Federico Costa e Emanuele De Simone

Design Document

Deliverable:	DD
Title:	Design Document
Authors:	Federico Costa e Emanuele De Simone
Version:	1.0
Date:	December-2025
Download page:	https://github.com/EmanueleDeSimone04/CostaDeSimone
Copyright:	Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

Contents

Table of Contents **3**

List of Figures **5**

List of Tables **5**

1 Introduction **6**
1.1 Purpose 6
1.2 Scope 6
1.3 Definitions, Acronyms, Abbreviations 7
1.4 Revision History 7
1.5 Reference Documents 7
1.6 Document Structure 7
1.6.1 Introduction 7
1.6.2 Architectural Design 7
1.6.3 User Interface Design 7
1.6.4 Requirements Traceability 7
1.6.5 Implementation, Integration and Test Plan 8
1.6.6 Effort Spent 8
1.6.7 References 8

2 Architectural Design **9**
2.1 Overview 9
2.2 Component View 10
2.2.1 Client Tier 10
2.2.2 Application and Data Tiers 12
2.3 Deployment View 15
2.4 Runtime View 15
2.4.1 [UC1] - Register Account 16
2.4.2 [UC2] - Log In 17
2.4.3 [UC3] - Log Out 18
2.4.4 [UC4] - Enter as Guest 18
2.4.5 [UC5] - Record Trip 19
2.4.6 [UC6] - View Recorded Trips 19
2.4.7 [UC7] - Insert and Validate BP Manually 20
2.4.8 [UC8] - Insert BP Automatically 21
2.4.9 [UC9] - View previously inserted BPs 21
2.4.10 [UC10] - Publish BP 22
2.4.11 [UC11] - Search EBPs between Origin and Destination 23
2.4.12 [UC12] - Select Another EBP from the List 23
2.4.13 [UC13] - View BP Details of the Selected EBP 23
2.5 Synchronizer 24
2.6 Component Interfaces 25
2.6.1 Client-Side Interfaces 25
2.6.2 Server-Side Interfaces 27
2.6.3 External Interfaces 29
2.7 Selected Architectural Styles and Patterns 29
2.7.1 3-tier Architecture with Fat Client 29
2.7.2 MVC Pattern 29

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

2.7.3 Listener Pattern . . . 30

3 User Interface Design . . . 31
3.1 Overview . . . 31
3.2 User Flow: Guest . . . 32
3.2.1 Login / Enter as Guest . . . 33
3.2.2 Registration . . . 34
3.2.3 Explore Bike Paths - Guest . . . 35
3.2.4 EBP and BP Details - Guest . . . 36
3.3 User Flow: Registered User - Trips . . . 37
3.3.1 Login and Home - Registered User . . . 37
3.3.2 Record Trip view . . . 38
3.3.3 Trips List . . . 39
3.3.4 Trip Details . . . 40
3.4 User Flow: Registered User - BPs . . . 40
3.4.1 Your Bike Paths . . . 41
3.4.2 Choose Insertion Mode . . . 42
3.4.3 Manual BP Insertion . . . 43
3.4.4 Automatic BP Recording . . . 44
3.4.5 Confirmation of Automatically Retrieved Data . . . 45
3.4.6 BP Details and Publish . . . 46

4 Requirements Traceability . . . 47

5 Implementation, Integration Test Plan . . . 48
5.1 Overview . . . 48
5.2 Implementation Plan . . . 48
5.3 Features Identification . . . 48
5.4 Integration Strategy . . . 49
5.4.1 Integration Approach . . . 49
5.4.2 Integration Steps . . . 50
5.5 System Testing Strategy . . . 58
5.6 Additional Specifications on Testing . . . 58

6 Effort Spent . . . 59

7 References . . . 60
7.1 References . . . 60
7.2 Used Tools . . . 60

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

List of Figures

2.1 High-level view of the architecture . . . 9
2.2 High-level view of the fat client architecture . . . 9
2.3 Client Component diagram . . . 12
2.4 Application and Data Tiers Component diagram . . . 13
2.5 API Manager Component diagram . . . 14
2.6 Deployment diagram . . . 15
2.7 UC1 . . . 16
2.8 UC2 . . . 17
2.9 UC3 . . . 18
2.10 UC4 . . . 18
2.11 UC5 . . . 19
2.12 UC6 . . . 19
2.13 UC7 . . . 20
2.14 UC8 . . . 21
2.15 UC9 . . . 21
2.16 UC10 . . . 22
2.17 UC11 . . . 23
2.18 UC12 . . . 23
2.19 Trips synchronization . . . 24
2.20 Login synchronization . . . 25
3.1 High-level user flows for guests and RUs in the BBP UI. . . . 32
3.2 Access view with login, registration and guest entry. . . . 33
3.3 Registration view for creating a new RU account. . . . 34
3.4 Explore Bike Paths view for guest users. . . . 35
3.5 Details of a selected EBP and its BPs for a guest user. . . . 36
3.6 Home view for registered users. . . . 37
3.7 Record Trip view for registered users. . . . 38
3.8 "Your Trips" view listing recorded trips. . . . 39
3.9 Details view for a recorded trip. . . . 40
3.10 "Your Bike Paths" view listing the BPs created by the RU. . . . 41
3.11 View for selecting the insertion mode of a new BP. . . . 42
3.12 Manual insertion view for a new BP. . . . 43
3.13 Automatic recording view for a new BP. . . . 44
3.14 Confirmation view for data collected during automatic BP recording. . . . 45
3.15 Details and publication view for a BP. . . . 46
5.1 Integration Step 1: Local DB and DBMS Integration. . . . 50
5.2 Integration Step 2: Users Manager and DBMS Integration. . . . 51
5.3 Integration Step 3: API Manager and Users Manager Integration. . . . 52
5.4 Integration Step 4: API Manager and Trip Saver Integration. . . . 53
5.5 Integration Step 5: BP Saver and DBMS Integration with EGAS (stub). . . . 54
5.6 Integration Step 6: EBPs Manager and DBMS Integration with EGAS (stub). . . . 55
5.7 Integration Step 7: End-to-End Integration with Client-side Components and Synchronizer. 57

List of Tables

1 Requirements Traceability . . . 47

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

1 Introduction

In recent years, cycling has become an increasingly popular and sustainable means of transportation. However, cyclists often face difficulties in finding accurate and up-to-date information about BPs. Existing data are usually scattered, incomplete, or not maintained, making it hard to evaluate the quality, safety, and usability of available routes. This lack of reliable information affects both occasional and regular cyclists, who may encounter poorly maintained paths, obstacles, or unsuitable routes without prior notice.

The **Best Bike Paths (BBP)** system is conceived to address these issues by providing a digital platform that collects, organizes, and displays data about BPs. Through manual input and automatic acquisition from mobile sensors, BBP builds a shared, up-to-date map of cycling routes enriched with statistics and environmental data, supporting users in discovering and maintaining the best paths for their journeys.

1.1 Purpose

The purpose of this Design Document (DD) is to describe the architecture and design of the **Best Bike Paths (BBP)** system as defined in the Requirement Analysis and Specification Document (RASD).

This document refines the functional and non-functional requirements into a concrete software structure composed of components, connectors, interfaces and deployment elements. It provides developers and maintainers with a clear blueprint on how BBP is organized, how its part interact and which design choices have been made to satisfy the identified requirements.

1.2 Scope

As specified in the Requirement Analysis and Specification Document (RASD), BBP is a software system that allows users to create and browse a catalog of BPs, that is a path in which there is an actual bike track or a street with few cars and a speed limit suitable for bikes.

For this purpose RUs can record their trips, storing various data like distance covered, average speed and, when available, also meteorological information retrieved from an XMS.

RUs can insert in the system information about the paths, like their status or the presence of obstacles, either using the manual mode, specifying the name of the streets composing the path with their relative status or the automated mode, in which BBP acquires data from the mobile device while the user is biking. If willing to, the RUs can also publish the information entered.

More specifically, the automated mode can guess if the users are biking depending on their speed, reconstructs the path with the GPS information and, thanks to the accelerometer and gyroscope of the mobile device, can also detect the presence of obstacles such as potholes. Since this mechanism can produce false positives, users must always confirm or correct the information acquired through this mode.

Finally, users, even non-registered ones, can submit an origin and a destination to BBP in order to visualize all paths between these two points that have been previously published by other RUs with all the related information. If multiple paths exist, they are shown according to a score, which is a parameter computed by the system based on the condition of the path and how effective it is in letting users reach their destination.

This Design Document focuses on how the BBP system is organized internally in order to support all the functionalities described above. In particular, it defines the main architectural components, their responsibilities and interactions, the integration with the XMS and the EGAS and the use of persistent storage for trips, BPs and EBPs. The functional scope is exactly the one defined in the RASD. Here the emphasis is on the software architecture that realizes it.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

1.3 Definitions, Acronyms, Abbreviations

Acronym	Definition
BBP	Best Bike Paths
BP	Bike Path
BPs	Bike Paths
RU	Registered User
RUs	Registered Users
XMS	External Meteorological Service
GPS	Global Positioning System
EGAS	External Geo-spatial Analysis System
EBP	Explorable Bike Path
UI	User Interface
DB	Database
MVC	Model View Controller
DBMS	Data Base Management System

1.4 Revision History

Version 1.0: initial release

1.5 Reference Documents

- The Requirement Engineering and Design Project specification document A.Y. 2025-2026.
- Slides of the Software Engineering 2 course

1.6 Document Structure

This document is organized into seven chapters, each describing a specific aspect of the BBP system design.

1.6.1 Introduction

This chapter introduces the BBP system and this Design Document. It recalls the problem context, states the document purpose and scope, provides the main definitions and lists the reference documents.

1.6.2 Architectural Design

This chapter describes the overall architecture of BBP. It presents the component-and-connector view, the main interfaces, the data storage design, and the deployment architecture. It also highlights major architectural decisions and the adopted architectural styles.

1.6.3 User Interface Design

This chapter provides an overview of the user interface structure, describing the main pages, navigation flows, and how they interact with the underlying services.

1.6.4 Requirements Traceability

This chapter relates the architectural and design elements to the goals and requirements defined in the RASD, ensuring coverage and traceability.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

1.6.5 Implementation, Integration and Test Plan

This chapter outlines how the system will be implemented and integrated, and provides a high-level plan for testing the designed components and interfaces.

1.6.6 Effort Spent

This chapter reports the time spent by each team member on the DD preparation.

1.6.7 References

This chapter lists all sources, standards and external materials referenced in the DD.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

2 Architectural Design

2.1 Overview

The architecture chosen for the BBP app is a 3-tier architecture:

- **Client Tier**: the part of the application installed on the user's mobile device, it handles the presentation and the logic of managing the information retrieved by the GPS and the sensors. This kind of data is stored on a local data layer that is synchronized with the central server as soon as an internet connection is available. This tier communicates with the application tier through a RESTful API
- **Application Tier**: the tier in which the business logic of the application is implemented . It also includes a component managing the requests coming from the clients, translating them from RESTful calls to internal components calls
- **Data Tier**: the tier in which data is stored and accessed

Here follow two pictures depicting a very high level view of the architecture and the client structure:

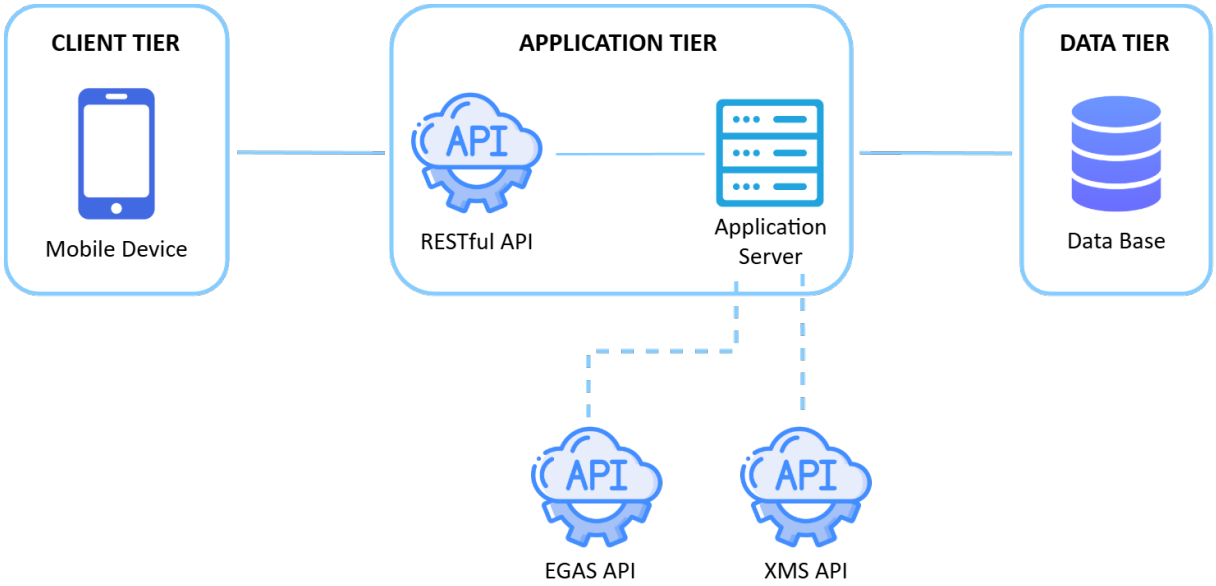


Figura 2.1: High-level view of the architecture

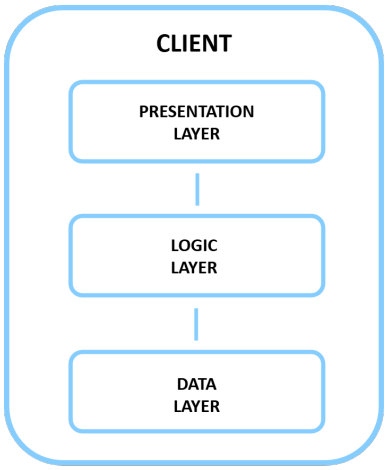


Figura 2.2: High-level view of the fat client architecture

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

Note that the application server interacts with two external APIs that offer the services presented in the RASD document:

- **EGAS**: it is needed to check that a BP is composed of an actual list of adjacent streets, when inserted manually, it generates a list of streets given a list of coordinates, and it can also generate maps of BP
- **XMS**: it is needed to retrieve meteorological information when recording a trip

It has been chosen a fat client architecture because it is strictly needed to manage locally the data retrieved by the sensors and the GPS efficiently, and also because it can be very useful to record trips without the need of an internet connection when users are out in zones, like the countryside, where the connection can be poor or unstable.

The 3-tier architecture has then been chosen because it makes easy and scalable the management of requests coming from different clients, with respect to mainly the publication of BPs and the visualization of previously published BPs. It brings also advantages in terms of modularity and maintainability.

More on the architecture will be discussed in the next sections of this chapter.

2.2 Component View

In this section, for each tier, there is a list of the components of BBP with the relative component diagrams.

2.2.1 Client Tier

The following components can be identified for the client tier:

- **Local DB**: a small DB on the mobile device in which the data regarding the recorded trips and the inserted BPs by the RU are stored
- **Trip Manager**: this component is responsible for interacting with the GPS to record trips and saving them locally on the device. If the internet connection is on and the RU gives their confirmation, this component retrieves meteorological information from the server (which in turn uses the XMS API). It also handles the trip navigation functionality that lets the RU consult the trip they previously recorded and are available on the local DB
- **Bike Paths Manager**: this component is in general responsible for inserting BPs in BBP:
  - **Manual Mode**: the RU submits a list of streets that is then sent to the server, which then calls the EGAS API to verify its validity and returns a response to the client
  - **Automated Mode**: in this case the app relies on the GPS to build a list of coordinates (latitude and longitude) and the sensors to detect the potholes. The list is then sent to the server that calls the EGAS API to build the corresponding list of streets

  Therefore, given these two cases, the component implements all the logic that is needed to build BPs and then store them in the local DB. It also handles the BP navigation functionality that lets the RU consult the BPs they previously inserted and are available on the local DB

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

- **Synchronizer**: this component silently updates the server on the changes in the local DB whenever the internet connection becomes available (If the storing operations mentioned above are done while the internet connection is active, the component in charge directly updates the server). It can also modify the local DB based on the server DB state, in fact misalignments are handled in this way:
  - new trips present on the local DB are always added to the server DB
  - since BPs and trips are immutable objects (since the eventual corrections done in automated mode are performed before saving the BP on the DB and the publication state of a BP is something living only on the server), if the same trip/BP has two different versions with different information on the server and on the local DB, then the synchronizer updates the local DB with the version on the server
  - in general if there is some information about the user in the server DB that is not present in the local DB, the component updates the local DB with the information on the server (this is helpful when RUs log in on a new device, because it lets them restore all of their personal information)
- **View**: this component is responsible for implementing the presentation layer, therefore, it must manage the inputs of the user and communicate with all the other components and the server, to modify the state of the application and show the changes to users (More on the Model View Controller pattern in section 2.6)
- **Application Manager**: this component supports the view component in all the operation that do not have a dedicated component on the client, for example log in requests, registration requests and all the operation related to public BPs. It is needed because the view merely displays changes and processes the users' inputs. This component handles the API calls to the server for the operation mentioned and calls the view component once the server responds to let it display the changes (Again, more on the MVC pattern later)

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

Here is the component diagram showing all the client's components and how they are connected through the interfaces they offer:

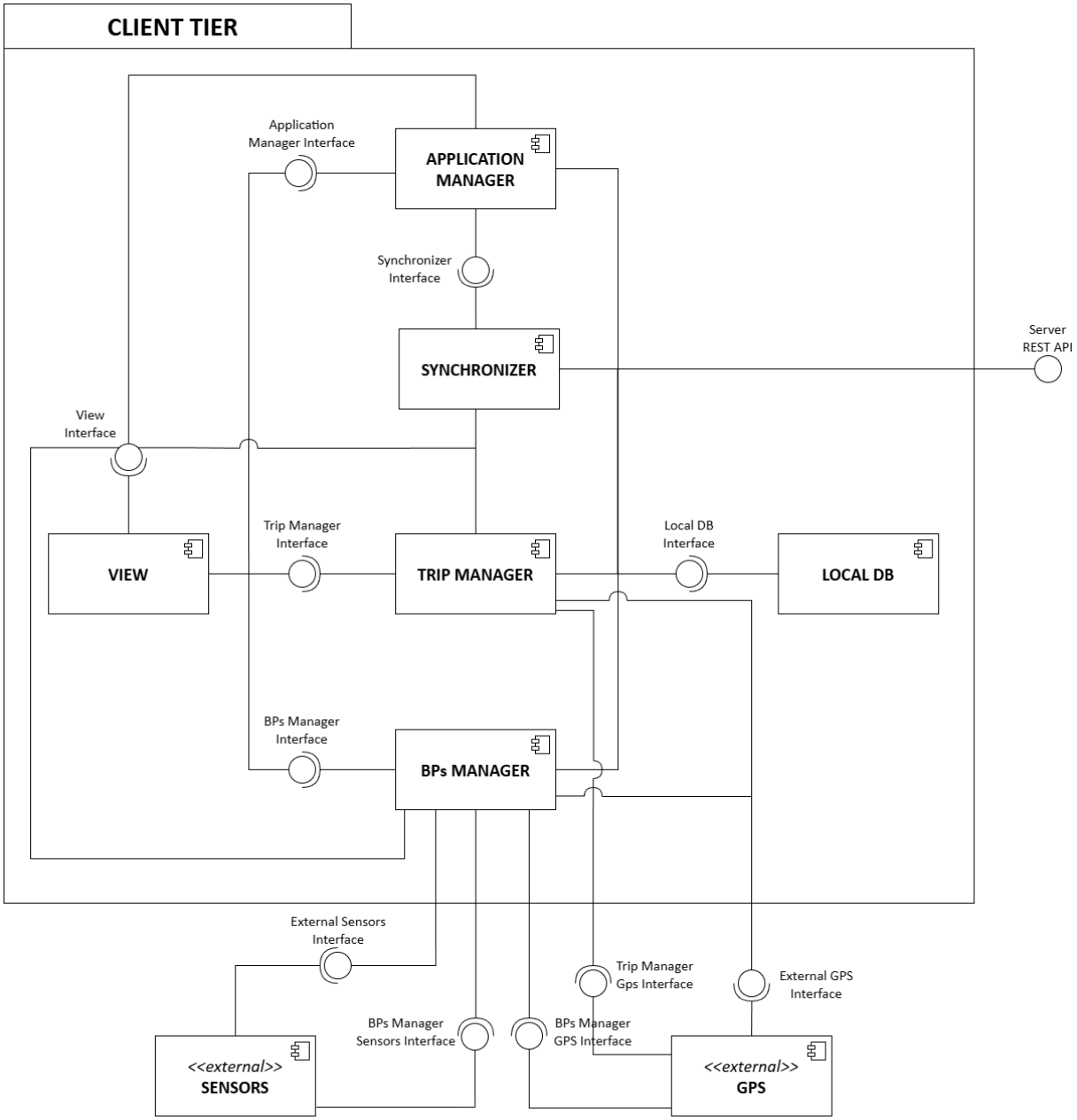


Figura 2.3: Client Component diagram

2.2.2 Application and Data Tiers

The following components can be identified for the application and data tiers:

- **API Manager**: this component handles the requests coming from the clients and, depending on the request, it forwards it to the suited component through its interface
- **Users Manager**: this component is responsible for registration operations, log ins and everything that concerns user identification
- **DBMS**: the component that stores all BBP's data

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

- **EBPs Manager**: this component manages the publication of a BP and the consequent creation of a new EBP or the integration in an already existing EBP, according to the rules described in the RASD. It also handles the BP search functionality, managing the maps generation using the EGAS API
- **Trip Saver**: this component is responsible for just updating the server DB with trips that have just been inserted in the client with an internet connection, or that have been reported to the server by the synchronizer component client-side. It also interacts with the XMS API to retrieve meteorological information if the the trip has just been inserted with an internet connection.
- **BP Saver**: the equivalent of Trip Saver for BPs, the only difference being that it interacts with the EGAS API to verify if a manually inserted BP has a correct list of streets and to retrieve a list of streets given a list of coordinates.

Here is the component diagram showing all the application and data tiers components and how they are connected through the interfaces they offer:

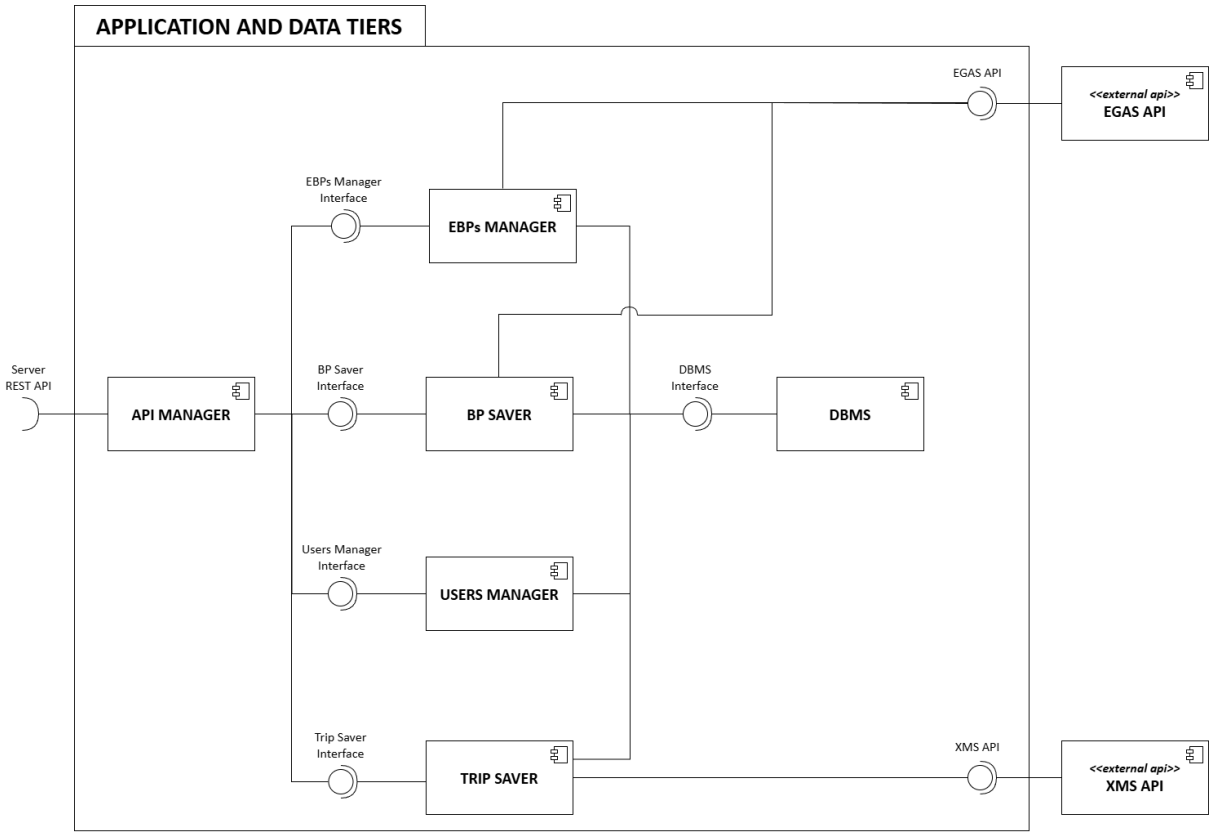


Figura 2.4: Application and Data Tiers Component diagram

Note that the full component diagram is just given by connecting the two diagrams above through the Server REST API.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

Since some of these components will run on different servers, as shown in the next section, the API Manager component is actually divided into two independent sub-components forwarding the requests to different components:

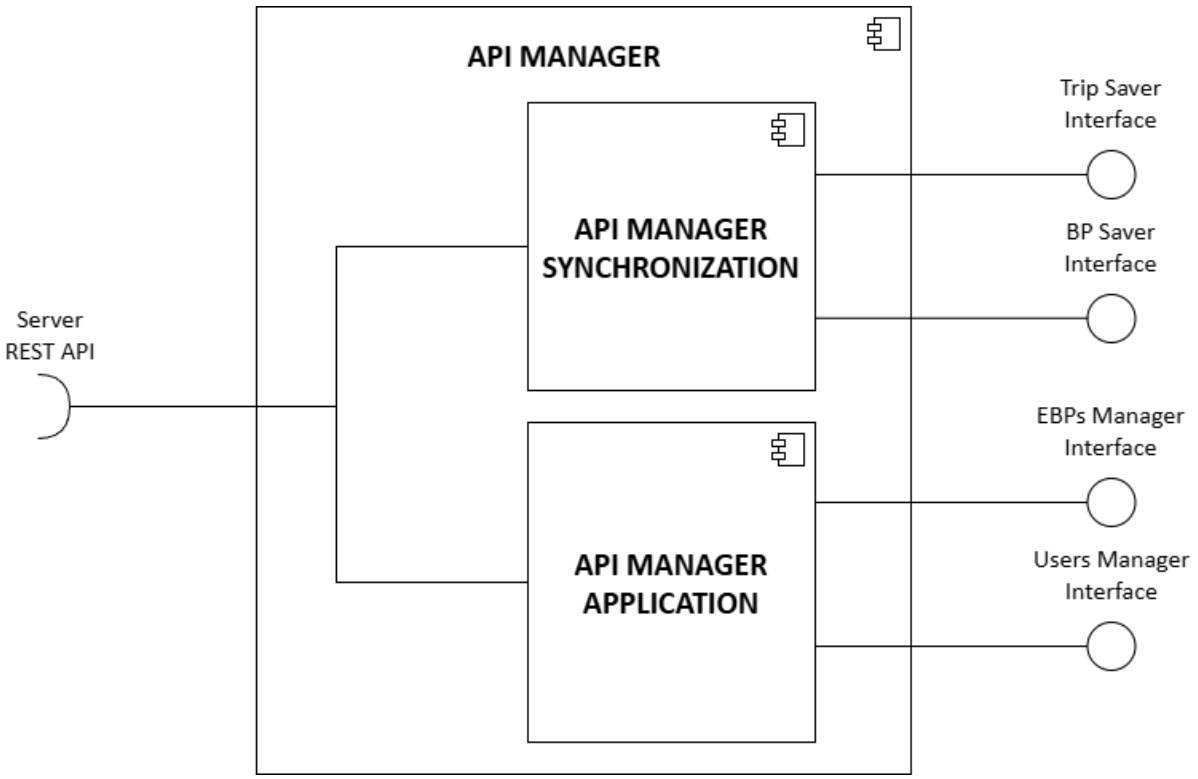


Figura 2.5: API Manager Component diagram

The names chosen are just the names of the servers on which the two different components are used.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

2.3 Deployment View

In order to guarantee the non-functional requirements listed in the RASD, the deployment shown in the following figure has been chosen:

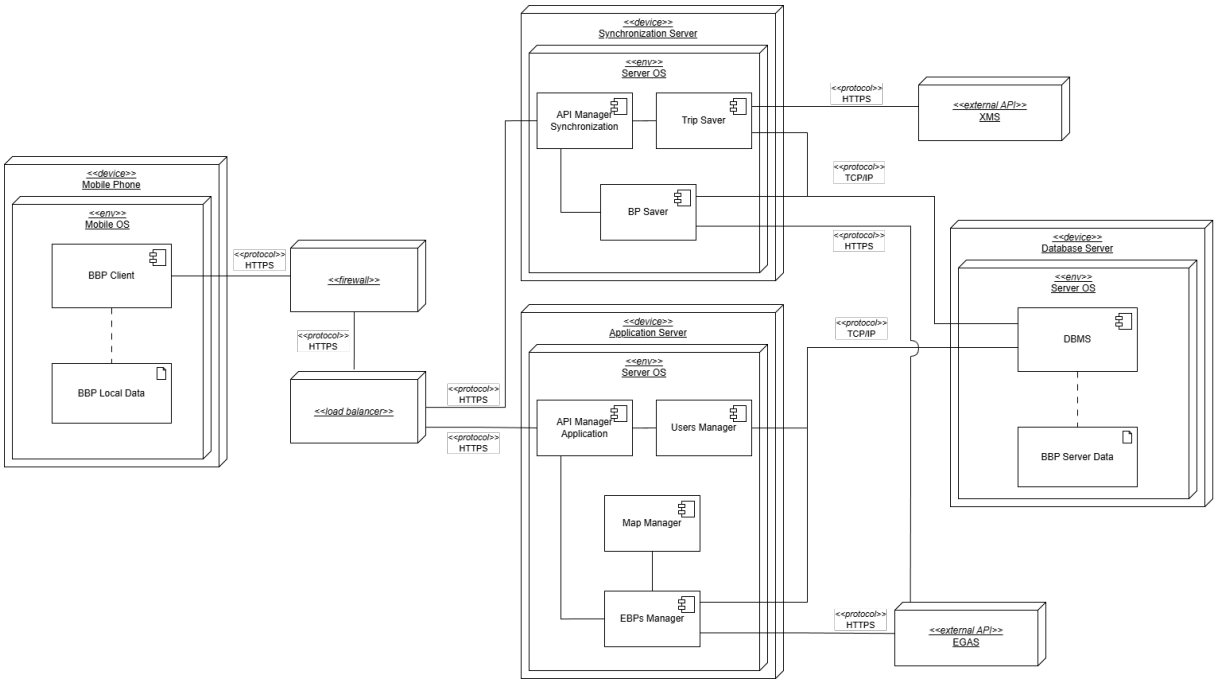


Figura 2.6: Deployment diagram

These are the main non functional requirements taken into consideration:

- **Availability**: in order to comply with the availability requirements, the logic regarding the saving of BPs and trips is handled on a different server than the logic handling the EBPs and the users. This is done because of independent scaling requirements, since the two different services scale in a different way because the synchronization operations handled by the *Synchronization Servers* are done each time the Internet connection becomes available on the client or when a RU enters a BP while online, while the *Application Servers* are needed whenever a user needs to search for an EBP or logs in (registers/enters as a guest), so less frequently. Therefore, these two kinds of servers are replicated in a different way, giving priority to the Synchronizer Server, and the load balancer redirects the request to the right server depending on the type of request and the current computational load of the server. Of course, to allow horizontal scaling, if these two servers are replicated, the database server too will need replication if necessary

- **Security**: security is guaranteed by the use of the HTTPS communication protocol and by a firewall put between clients and servers

- **Maintainability**: inherited by the 3-tier architecture and the definition of different modular components

2.4 Runtime View

In this section, all the use cases listed in the RASD are better described at a lower level using the component defined previously.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

2.4.1 [UC1] - Register Account

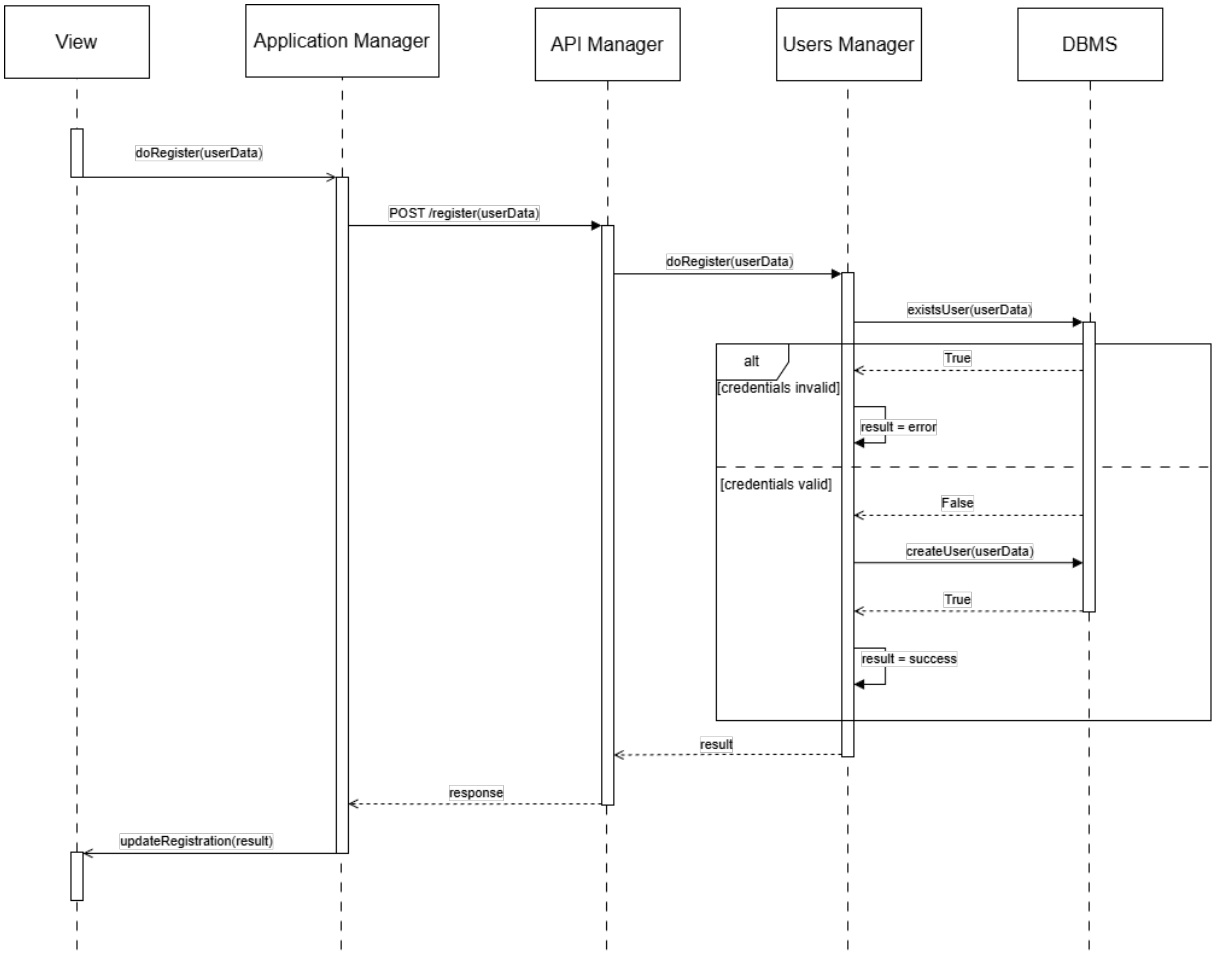


Figura 2.7: UC1

The figure shows the registration process flow. The view component, after receiving the input from the user, builds a JSON file with the first name, last name, username, email, and password called userData. It then calls the application manager component which, in turn, calls the API manager through an HTTPS POST with userData as the body of the request. The API manager then redirects the request to the suited component, that is the users manager component, that first verifies if the credentials inserted by the user are valid, and then creates the account in the DB. Depending on the result, the application manager finally updates the view according to the MVC pattern. Note that, in this case, error and success are the HTTPS codes representing the result of the operation.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

2.4.2 [UC2] - Log In

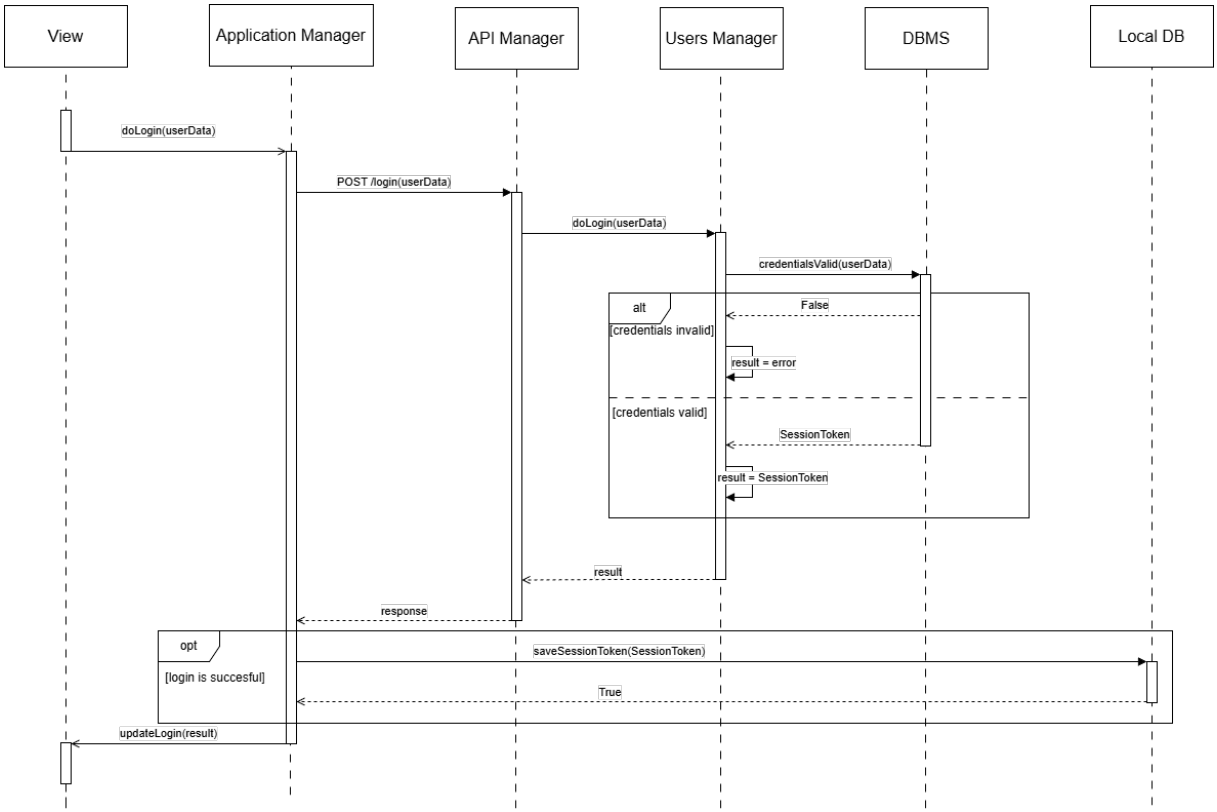


Figura 2.8: UC2

Similar to the registration with the difference being that, in this case, if the login is successful, a JSON Web Token is created to identify the user in later operations. The token is stored on the user's device and it is put in the HTTPS requests' headers each time an operation is performed while logged in to identify the RU. This step is omitted in the next sequence diagrams. The token expires after a week without being used.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

2.4.3 [UC3] - Log Out

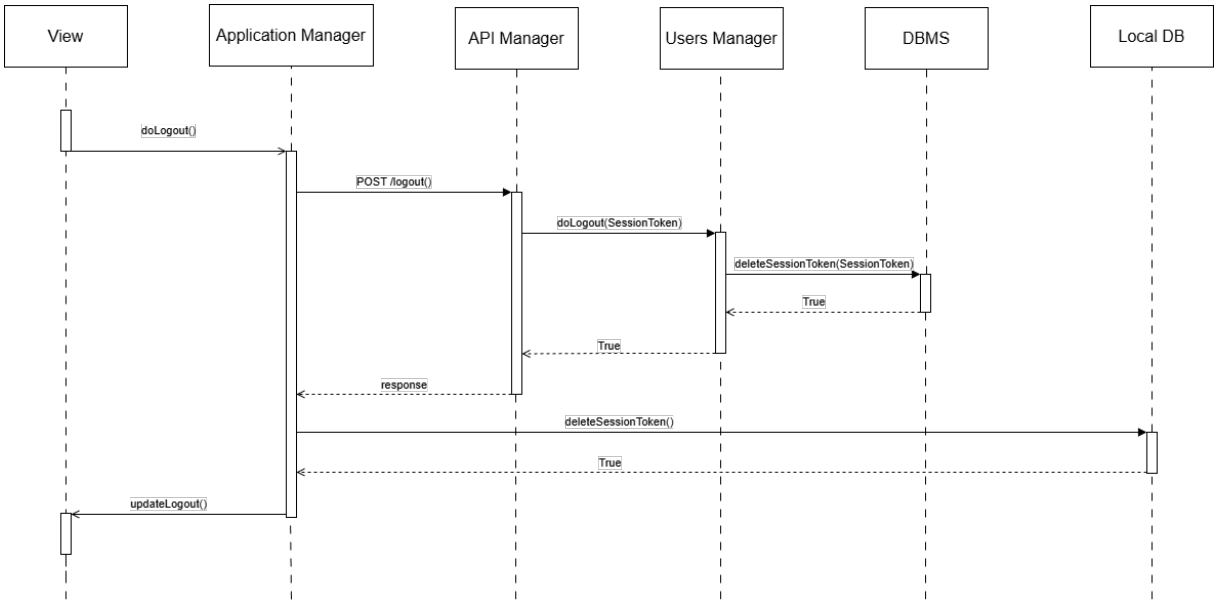


Figura 2.9: UC3

When a user logs out, the session token is deleted on the server and locally.

2.4.4 [UC4] - Enter as Guest

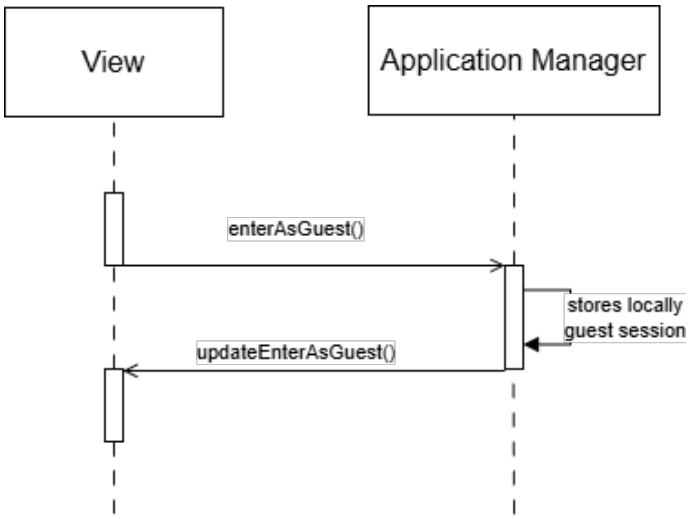


Figura 2.10: UC4

Entering as guests is an operation that does not involve the server, it simply redirects the user onto another screen designed for guests.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

2.4.5 [UC5] - Record Trip

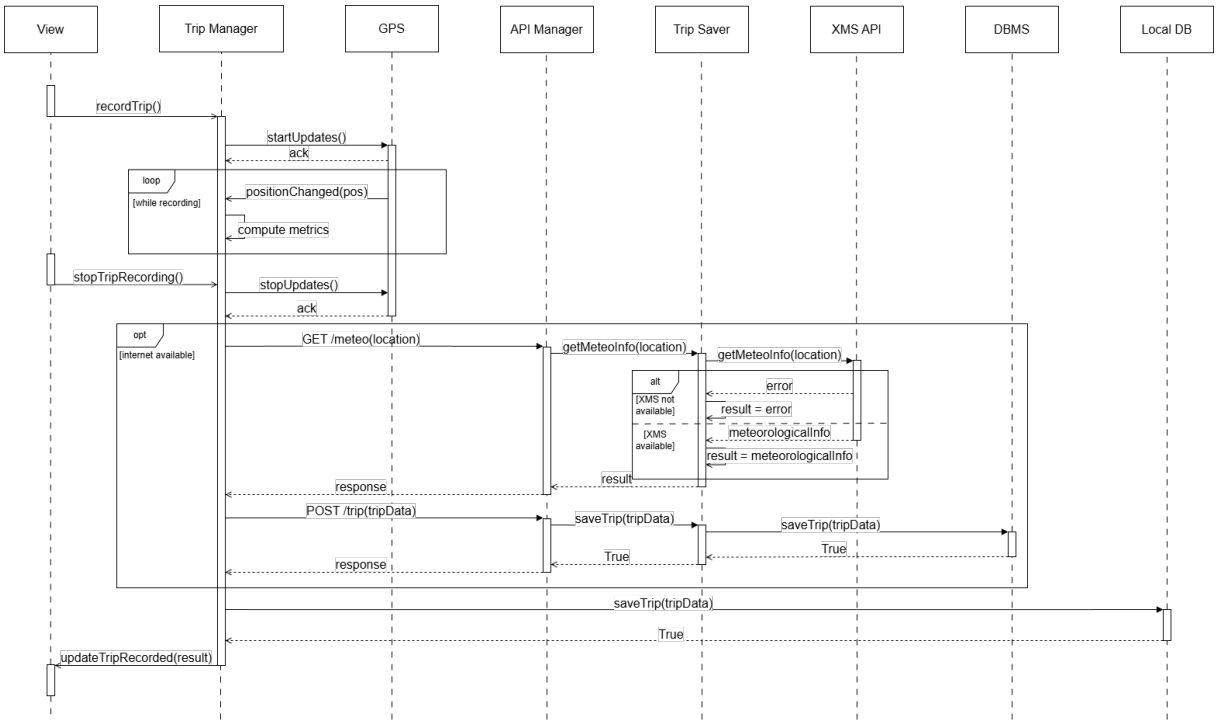


Figura 2.11: UC5

For readability reasons, it has been omitted that in the GPS loop if the trip manager component detects a speed exceeding 60 km/h, the trip recording is canceled, while if the position detected becomes constant, meaning that the speed is 0 km/h, the component pauses the recording, so it does not update the metrics anymore until the position detected changes.

2.4.6 [UC6] - View Recorded Trips

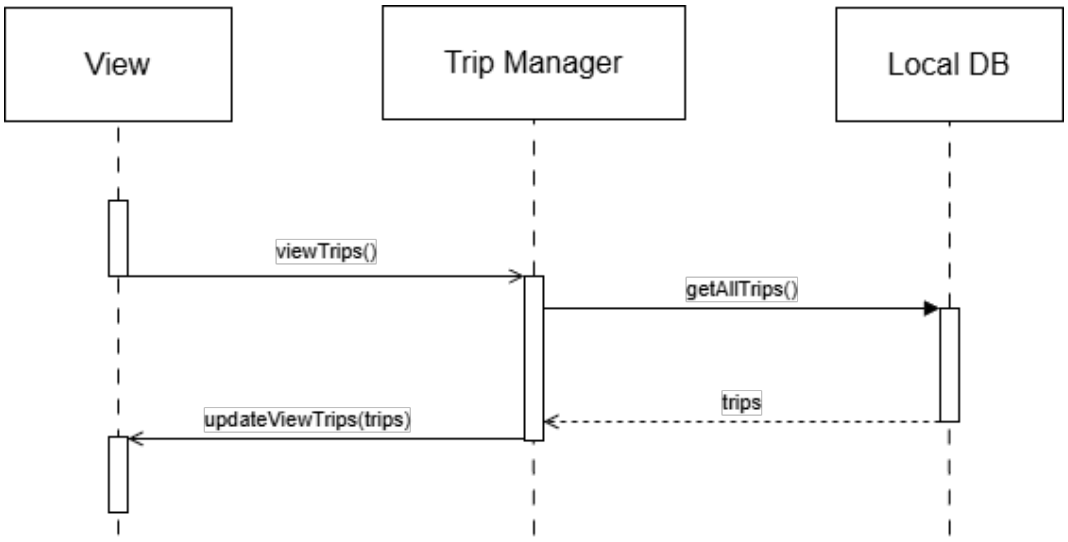


Figura 2.12: UC6

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

The fat client architecture here lets us perform this operation without calling the sever (so without an internet connection). The view component, once obtained the trips JSON file of all the RU's trips, handles the display of the list and of the specific trips once they are clicked on.

2.4.7 [UC7] - Insert and Validate BP Manually

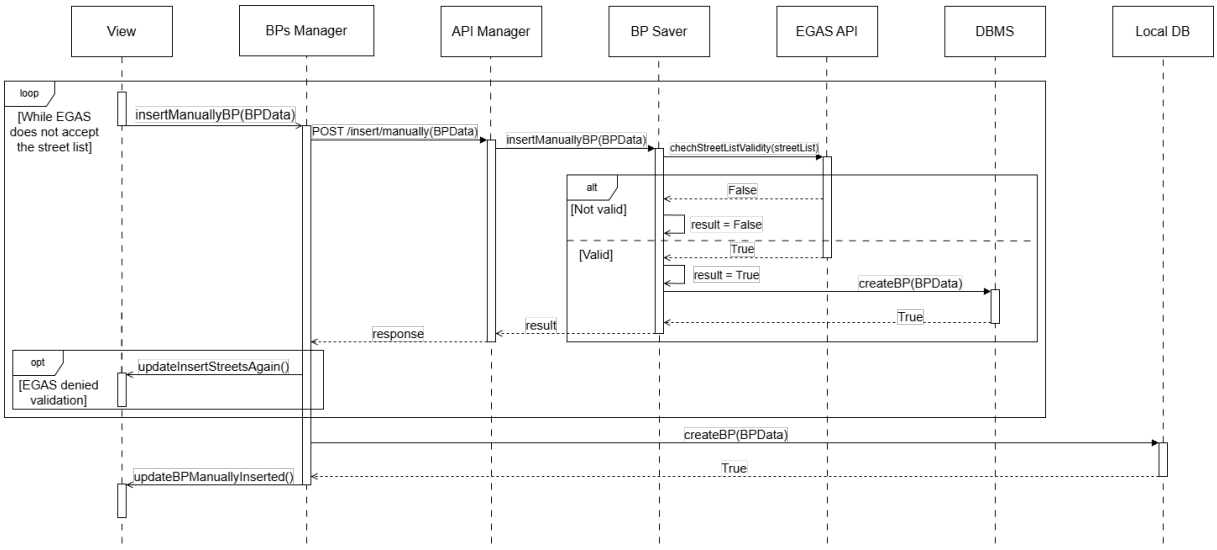


Figura 2.13: UC7

Note that, since the list of the streets selected by the RU must be validated by the EGAS, this operation can be performed only with an available internet connection.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

2.4.8 [UC8] - Insert BP Automatically

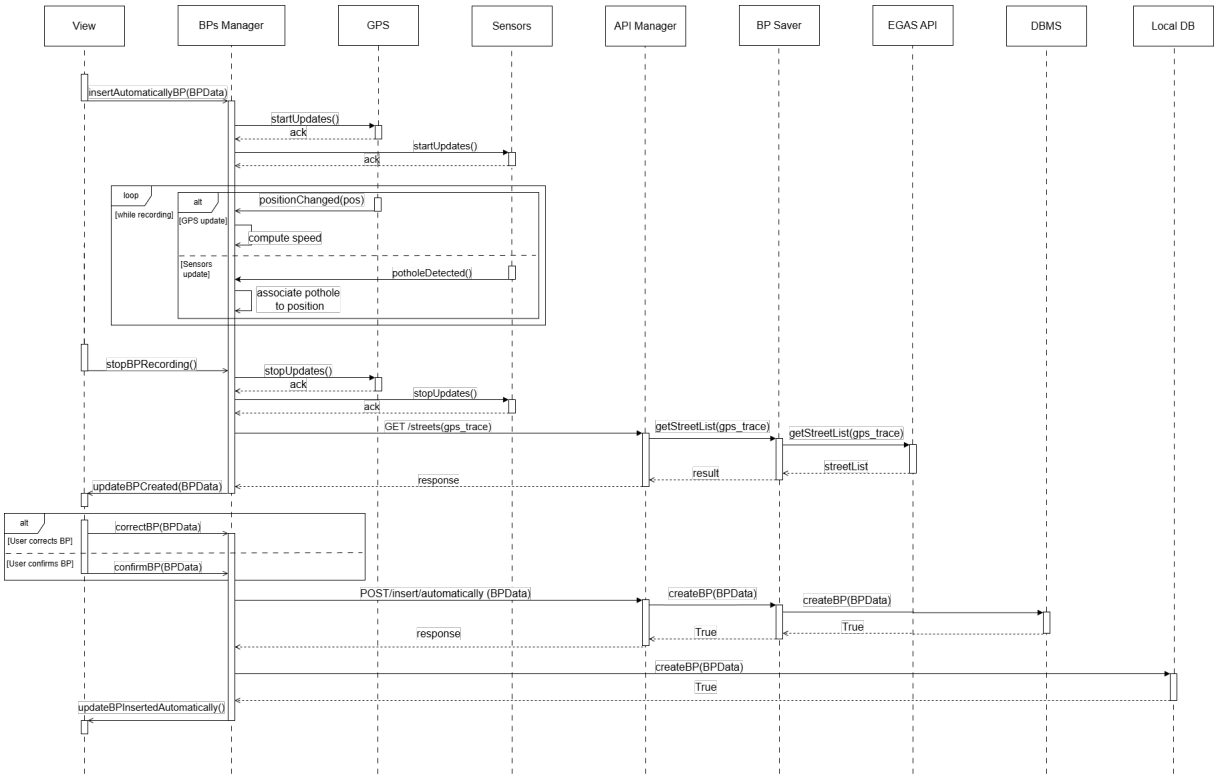


Figura 2.14: UC8

The same omission done in UC5 is done here in the exact same way. The pothole is associated to the last position reported by the GPS and saved in the BPs manager component. Again, like before, this operation requires an internet connection.

2.4.9 [UC9] - View previously inserted BPs

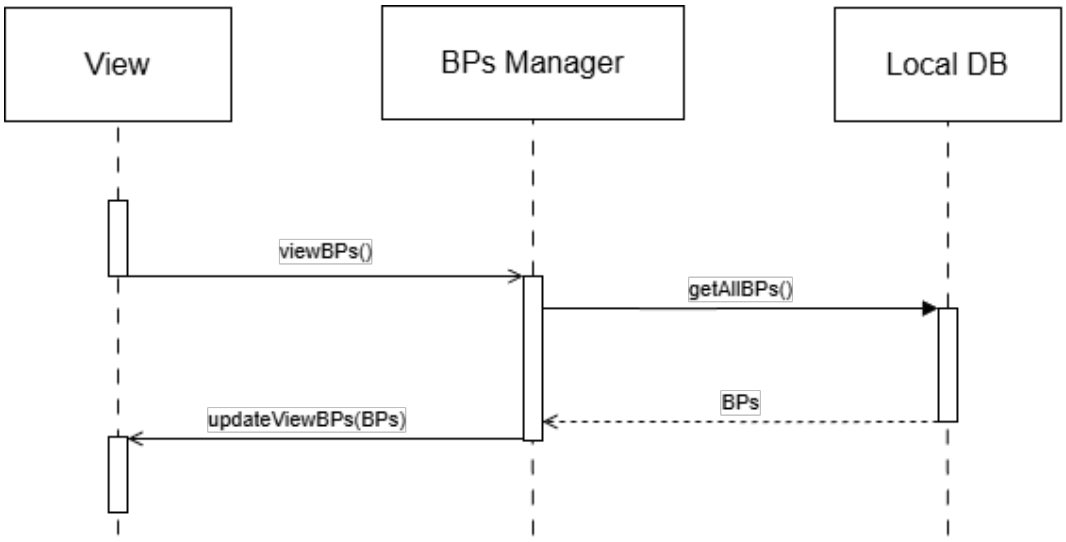


Figura 2.15: UC9

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

This use case is similar to UC6, in the sense that once the view has obtained the BPs to visualize, it handles the visualization part by itself.

2.4.10 [UC10] - Publish BP

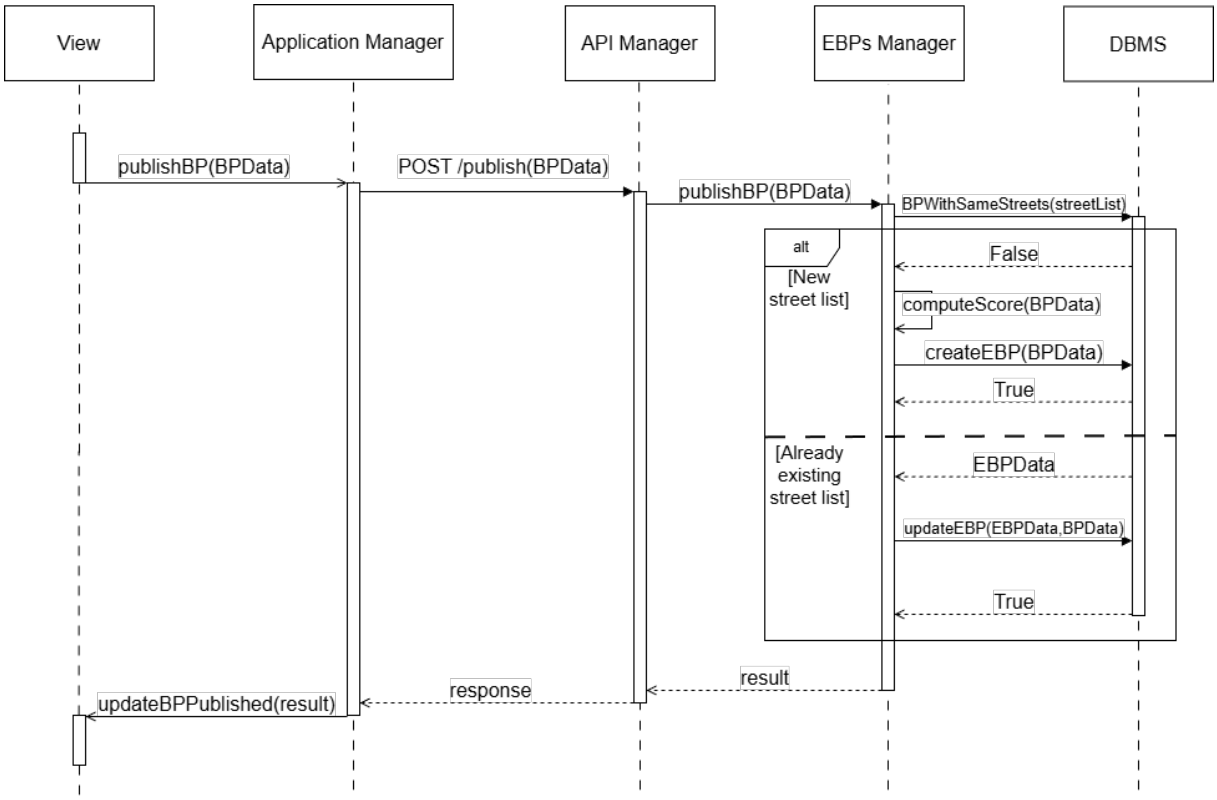


Figura 2.16: UC10

The only thing to highlight here is that, as specified before, since the operation is neither the insertion of a BP, nor the recording of a trip, the component handling the operation client-side is the application manager. As specified in the RASD, if an EBP with the same exact street list is already present in the DB, then the BP is integrated in it (updateEBP), otherwise a new EBP is created.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

2.4.11 [UC11] - Search EBPs between Origin and Destination

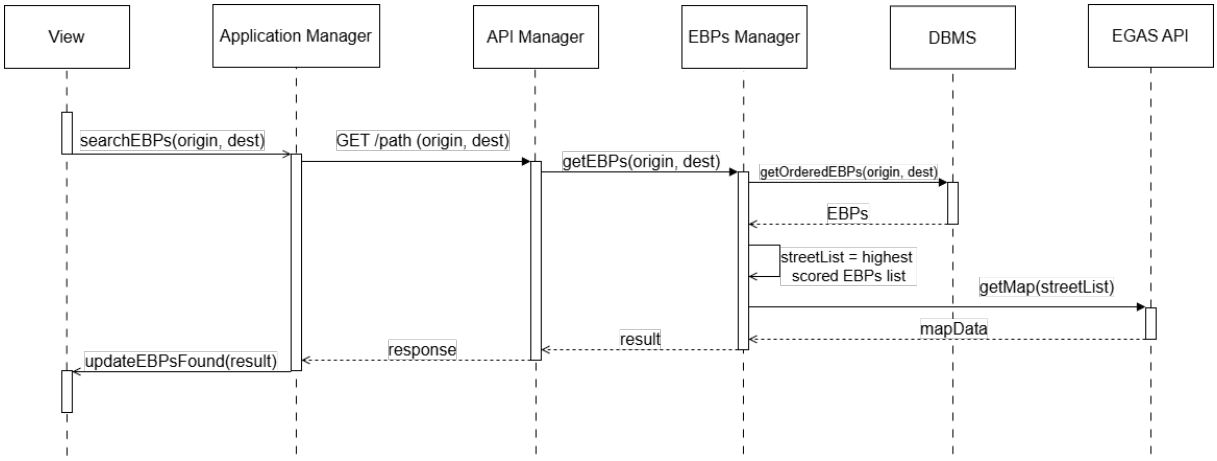


Figura 2.17: UC11

In this UC, BBP interacts with the EGAS API to retrieve the mapData of the highest scored EBP in order to let the view display it once the operation is over.

2.4.12 [UC12] - Select Another EBP from the List

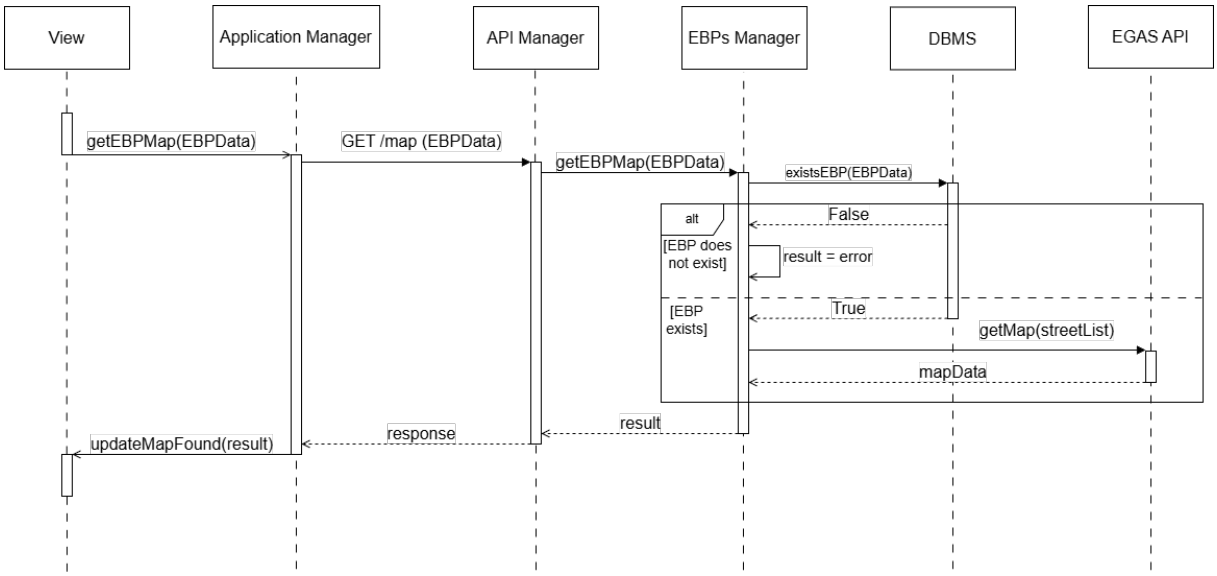


Figura 2.18: UC12

The information about the other EBPs is stored in the client while the user is looking for a BP between two points, the only thing needed to display another EBP is the map that, like before, is obtained from the EGAS API. Once the map needed has reached the client, the view handles its visualization and also the one of the list of the other EBPs (whose info is still in the client) like it is stated in the RASD.

2.4.13 [UC13] - View BP Details of the Selected EBP

In this case the view component handles on its own the entire operation, because it just needs to use the EBPs data that are currently available locally to display the details of the EBP selected. Since this can be

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

done only with an EBP whose map is being shown, also the visualization of the potholes is done locally, because it is just a matter of putting some pins (representing the potholes) over the image of the map that is already available to the client.

2.5 Synchronizer

This component is responsible for dealing with misalignments between the client and the server each time the internet connection becomes available (also when a user logs in on a new device). The following sequence diagram describes how the synchronizer handles the synchronization of the trips between client and server whenever the internet connection becomes available:

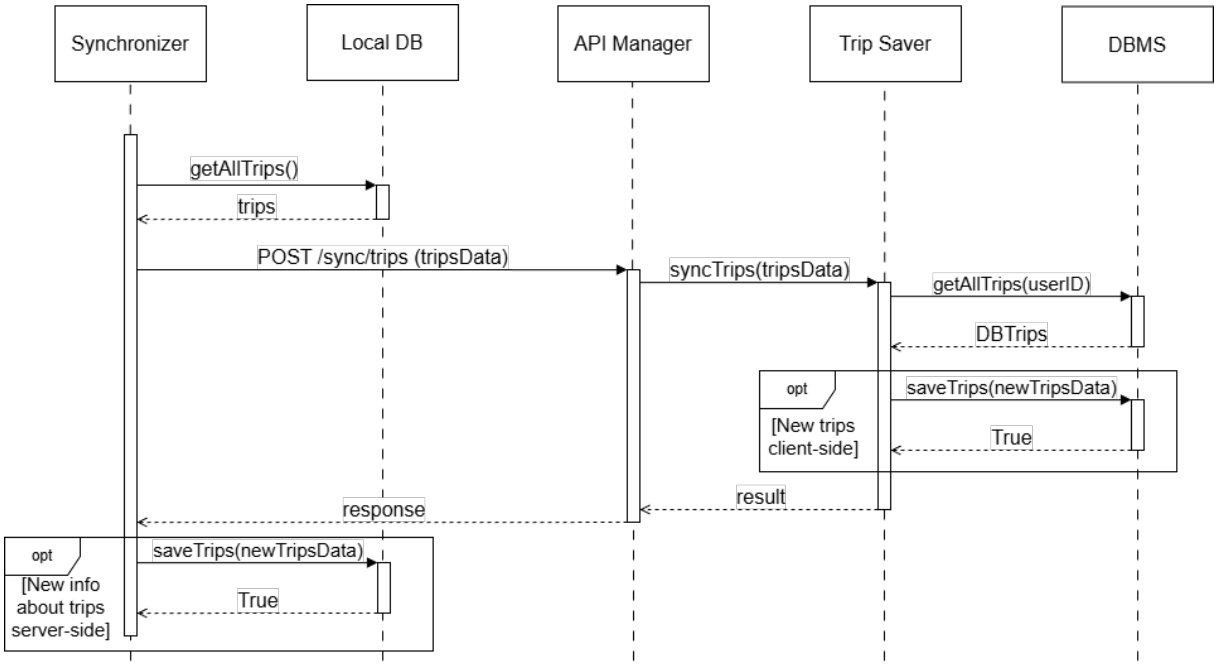


Figura 2.19: Trips synchronization

Finally, this sequence diagram shows how the synchronizer retrieves the data when RUs log in. It should be read with figure 2.8, in particular, the application manager component calls the synchronizer immediately before updating the view in the opt block:

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

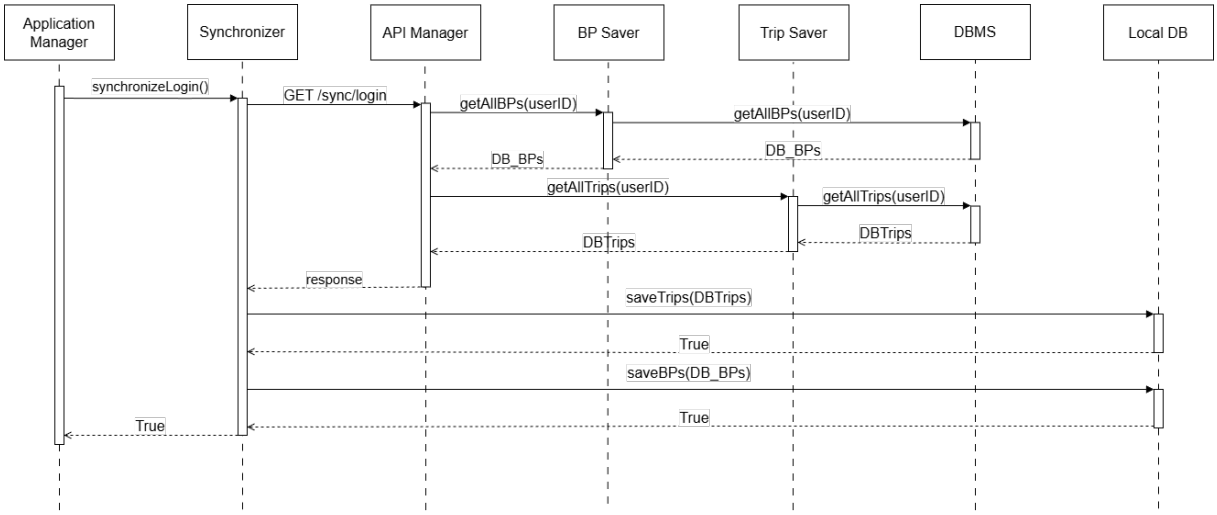


Figura 2.20: Login synchronization

2.6 Component Interfaces

Note that all objects labeled as Data are JSON format strings always containing all necessary information for their purpose, for example tripData has the user ID and all the information related to the trip, as described in the RASD.

2.6.1 Client-Side Interfaces

Local DB

- saveSessionToken(SessionToken)
- deleteSessionToken()
- saveTrip(tripData)
- getAllTrips()
- createBP(BPData)
- getAllBPs()
- saveTrips(tripsData)
- saveBPs(BPsData)

Trip Manager (Trip Manager Interface)

- recordTrip()
- stopTripRecording()
- viewTrips()

Trip Manager (Trip Manager GPS Interface)

- positionChanged(pos)

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

Bike Paths Manager (BPs Manager Interface)

- insertManuallyBP(BPData)
- insertAutomaticallyBP(BPData)
- stopBPRecording()
- correctBP(BPData)
- confirmBP(BPData)
- viewBPs()

Bike Paths Manager (BPs Manager GPS Interface)

- positionChanged(pos)

Bike Paths Manager (BPs Manager Sensors Interface)

- potholeDetected()

Synchronizer

- synchronizeLogin()

View

- updateRegistration(result)
- updateLogin(result)
- updateLogout()
- updateEnterAsGuest()
- updateTripRecording(result)
- updateViewTrips(trips)
- updateInsertStreetsAgain()
- updateBPManuallyInserted()
- updateBPCreated(BPData)
- updateBPInsertedAutomatically()
- updateViewBPs(BPs)
- updateBPPublished(result)
- updateEBPsFound(result)
- updateMapFound(result)

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

Application Manager

- doRegister(userData)
- doLogin(userData)
- doLogout()
- enterAsGuest()
- publishBP(BPData)
- searchEBPS(origin, dest)
- getEBPMap(EBPData)

2.6.2 Server-Side Interfaces

API Manager

- POST /register(userData)
- POST /login(userData)
- POST /logout
- GET /meteo(location)
- POST /trip(tripData)
- POST /insert/manually(BPData)
- GET /streets(gps_trace)
- POST /insert/automatically(BPData)
- POST /publish(BPData)
- GET /path(origin, dest)
- GET /map(EBPData)
- POST /sync/trips(tripsData)
- GET /sync/login

Users Manager

- doRegister(userData)
- doLogin(userData)
- doLogout(SessionToken)

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

DBMS

- existsUser(userData)
- createUser(userData)
- credentialsValid(userData)
- deleteSessionToken(SessionToken)
- saveTrip(tripData)
- createBP(BPData)
- BPWithSameStreets(streetList)
- createEBP(EBPData)
- updateEBP(EBPData, BPData)
- getOrderedEBPs(origin, dest)
- existsEBP(EBPData)
- getAllTrips(userID)
- saveTrips(tripsData)
- getAllBPs(userID)

EBPs Manager

- publishBP(BPData)
- getEBPs(origin, dest)
- getEBPMap(EBPData)

Trip Saver

- getMeteoInfo(location)
- saveTrip(tripData)
- syncTrips(tripsData)
- getAllTrips(userID)

BP Saver

- insertManuallyBP(BPData)
- getStreetList(gps_trace)
- createBP(BPData)
- getAllBPs(userID)

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

2.6.3 External Interfaces

EGAS API

- checkStreetListValidity(streetList)
- getStreetList(gps_trace)
- getMap(streetList)

XMS API

- getMeteoInfo(location)

Sensors

- startUpdates()
- stopUpdates()

GPS

- startUpdates()
- stopUpdates()

2.7 Selected Architectural Styles and Patterns

2.7.1 3-tier Architecture with Fat Client

As shown in the introduction of this chapter, the architecture chosen for BBP is a 3-tier architecture with fat client. BBP in fact requires the interactions with the device's sensors and GPS that, performance-wise, would be terrible if the client had only handled presentation. For instance, each time the GPS had reported a new position, the client would have had to send a message to the server to handle the logic of the path creation. This is the main reason why the client has a logic tier, but not the only one, because this architecture also lets the RU record trips in remote zones while not having an available internet connection, because there is no need to immediately contact the server. Therefore, in order to store trips recorded offline or to visualize previously inserted BPs or trips, the client also has a data layer.
With respect to the 3-tier architecture, this choice offers significant advantages in terms of modularity and scalability. The client tier and the application tier communicate through HTTPS messages, in fact the application tier exposes a RESTful API, and data is exchanged using JSON files put in the HTTPS requests/responses. The application tier is responsible for saving trips and BPs in the DB, publishing BPs, handling the EBPs and communicating with the external APIs. Finally, the data tier is used to store everything, from the BPs inserted, the EBPs published, to the trips, even the one inserted offline (that are eventually sent to the server thanks to the synchronizer). The communication between the data tier and the application tier is realized through SQL queries over TCP/IP, as shown in the deployment diagram.

2.7.2 MVC Pattern

The MVC architectural pattern has been chosen to realize BBP. In particular the presentation is completely handled by the view component, the state of the application, the model, is stored in the local DB and in the server-side DB, while the controllers are all the other components. In particular, the role of the controllers is to receive from the view the users' requests, check their validity and to read/update the

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

DBs (so the state of the application) to satisfy these requests. Then the controllers also have to inform the view about the changes performed to the model, that is why the view component exposes through its interface all the updateX methods.

2.7.3 Listener Pattern

To handle the presentation part of the application, the view component relies on the listener pattern. Listeners are registered to graphical components to monitor user inputs and, as soon as an event that is "listened to" happens (e.g. a user clicks on the record trip button), the associated action is performed by another component called Handler. This action can be of two types:

- **Graphical only**: the action does not modify the state of the application but only the state of the view (e.g. UC13);
- **Not only graphical**: the action reads or modifies the state of the application, so it involves some reads/updates in the DBs (e.g. UC2).

This pattern is useful because it is a standardized way to manage graphical interfaces.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

3 User Interface Design

This chapter describes the UI of the BBP mobile client application. Starting from the functionalities and scenarios defined in the RASD, it shows how users interact with the system through views, forms, lists and maps.

The description is organized by main user flows (guest users and RUs) and, for each flow, it presents the corresponding views together with their role, main elements and available actions. Where appropriate, the text is meant to be complemented by mockups that illustrate the layout and behaviour of the most relevant views.

3.1 Overview

BBP provides a graphical user interface through a mobile client application, installable on smartphones, which communicates with the BBP backend via HTTPS. The interface allows the user to submit information, visualize system outputs and interact with the functionalities defined in the requirements.

The UI is organized as a single mobile application that adapts its content depending on whether the user is authenticated as a RU or using the app as a guest. In both cases, views share a common layout composed of:

- **Top bar**: always visible at the top of the view. It contains the BBP logo, the view title and a openable three line menu giving access to entries such as *Home*, *Your Trips*, *Your Bike Paths*, *Explore Bike Paths* and either *Log In / Register* or *Log Out*, depending on the current session state.

- **Main content area**: the central part of the view, which changes according to the current functionality (e.g., trip recording, trips list, manual or automatic BP insertion, exploration of EBPs on the map, BP/EBP details).

- **Bottom bar**: a narrow bar at the bottom of the view containing legal information and links to documentation or support.

In the views that display maps (such as *Explore Bike Paths* and the EBP/BP Details views), the map content is obtained from the EGAS. The BBP backend queries EGAS and returns the corresponding map to the mobile client, which then displays it in the main content area.

The interface is designed to adapt to different screen sizes and orientations: navigation entries are accessible through a compact menu control in the top bar, while preserving the same logical structure and available actions.

When the user is in guest mode, entries reserved to RUs (such as *Your Trips* and *Your Bike Paths*) are hidden or disabled, so that only *Explore Bike Paths* and *Log In / Register* remain available.

Subsequent sections of this chapter describe the user interface following the main user flows: guest exploration of published BPs, trip management for RUs and BP management (manual and automatic insertion, confirmation and publication).

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

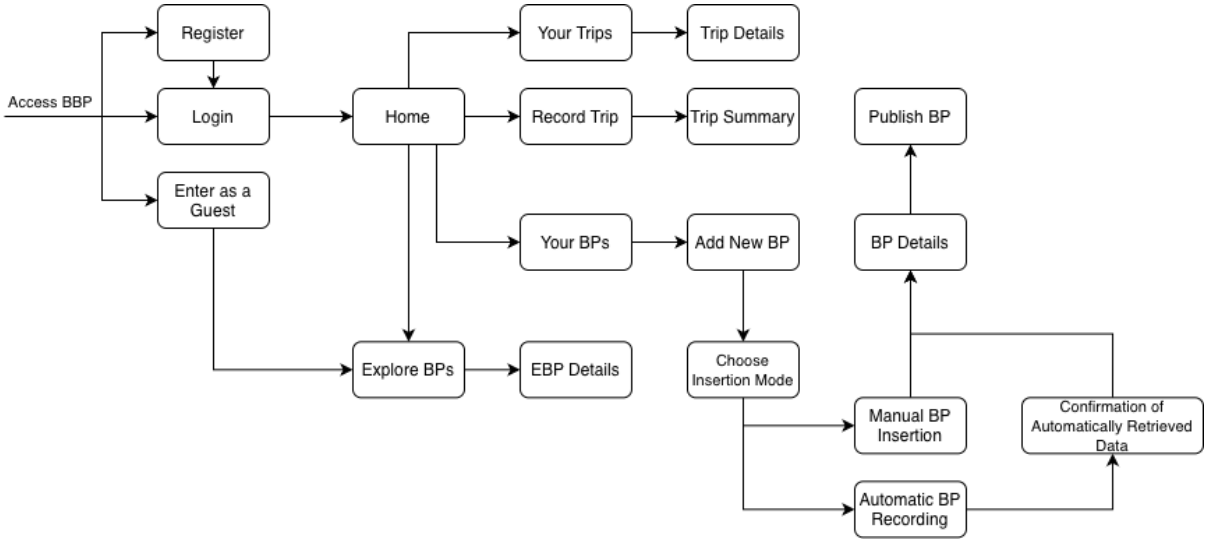


Figura 3.1: High-level user flows for guests and RUs in the BBP UI.

3.2 User Flow: Guest

Non-registered users can access BBP in *guest mode*. In this mode they do not need an account and are only allowed to explore BPs that have been published by RUs.

The guest flow starts from the login view, where the user can choose the *Enter as Guest* option. The system then grants guest access and navigates the user directly to the *Explore Bike Paths* view, which acts as the home screen for guests and is the only action enabled in the navigation menu. From there, the guest can specify an origin and a destination, request the list of available EBPs, visualize the highest-scored one on a map and inspect the details of the selected EBP and its associated BPs.

Figure 3.1 shows this flow on the left-hand side: *Access BBP* → *Enter as a Guest* → *Explore Bike Paths* → *EBP Details*.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

3.2.1 Login / Enter as Guest

The first view shown when accessing BBP is the *access view* (login view), which is shared by both registered users and guests. From here, a user can either log in with existing credentials, register a new account or continue in guest mode without creating an account.

The view contains:

- a login form with *username* and *password* fields and a *Log In* button;
- a *Register* link that opens the registration view for new users (see Section 3.2.2);
- a button *Enter as Guest* that lets the user proceed without creating an account.

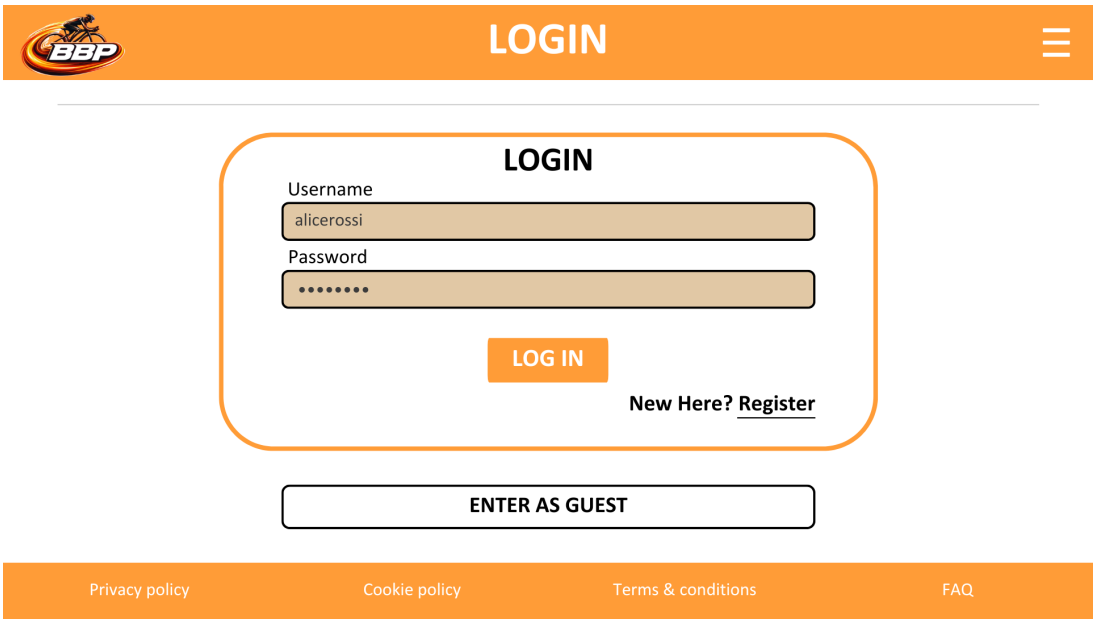


Figura 3.2: Access view with login, registration and guest entry.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

3.2.2 Registration

The *Registration* view is used by non-registered users to create a new RU account. It is accessed from the *access view* by activating the *Register* control.

The view contains a registration form with the following fields:

- *First name*;
- *Last name*;
- *Username*;
- *Email address*;
- *Password*.

Below the form, the view provides:

- a *Register* button, which submits the data to the backend to create the new account;
- a *Back to Login* control, which returns the user to the access view without creating an account.

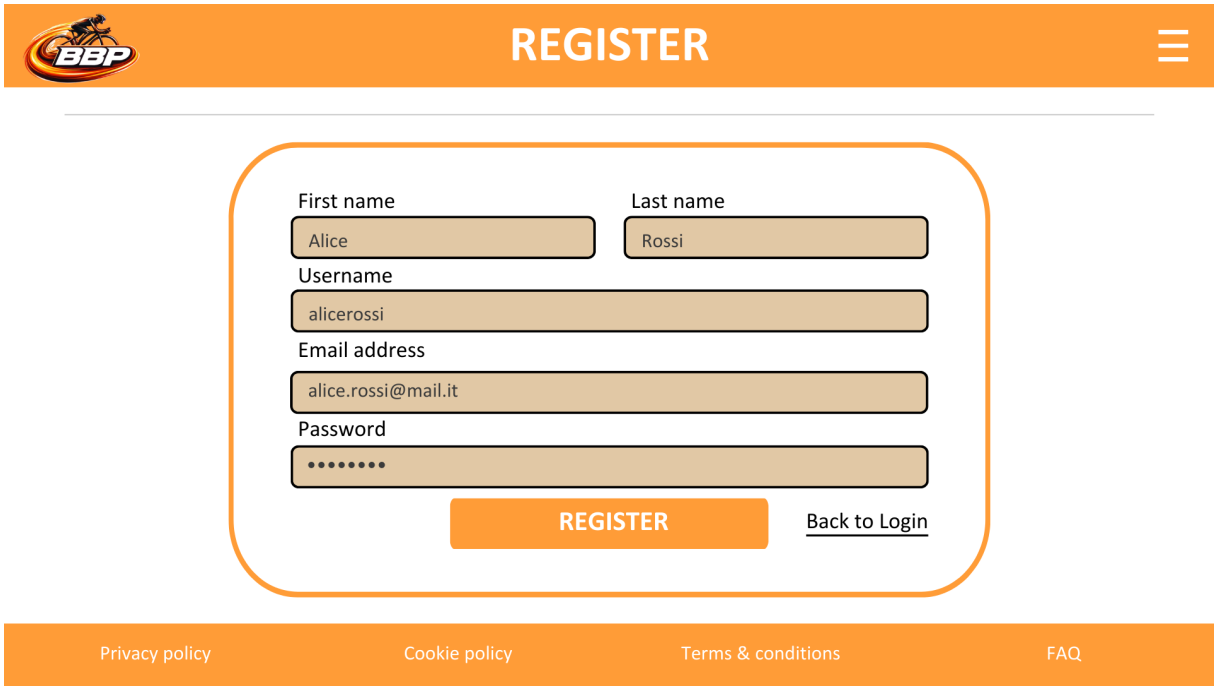


Figura 3.3: Registration view for creating a new RU account.

When the user presses *Register*, the interface validates the input and sends the registration request. If the data are invalid or if the chosen username or email are already associated with an existing account, the view displays an error message and highlights the problematic fields so that the user can correct them. If the registration succeeds, the user is created as a RU, is automatically authenticated and the application navigates to the home view.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

3.2.3 Explore Bike Paths - Guest

The *Explore Bike Paths* view is the main view available to guest users. Its purpose is to allow the user to search for EBPs between an origin and a destination and visualize the results.

The view is structured into two main areas:

- *Search panel*: a form where the user can enter the *origin* and the *destination* and start the search. A *Search* button triggers the request to the system, which retrieves all EBPs connecting the two points and orders them according to their score.
- *Results area*: composed of a map and a scrollable list of EBPs. After a successful search, the map shows the EBP with the highest score among the retrieved ones, while the list displays all of them in descending order of score.

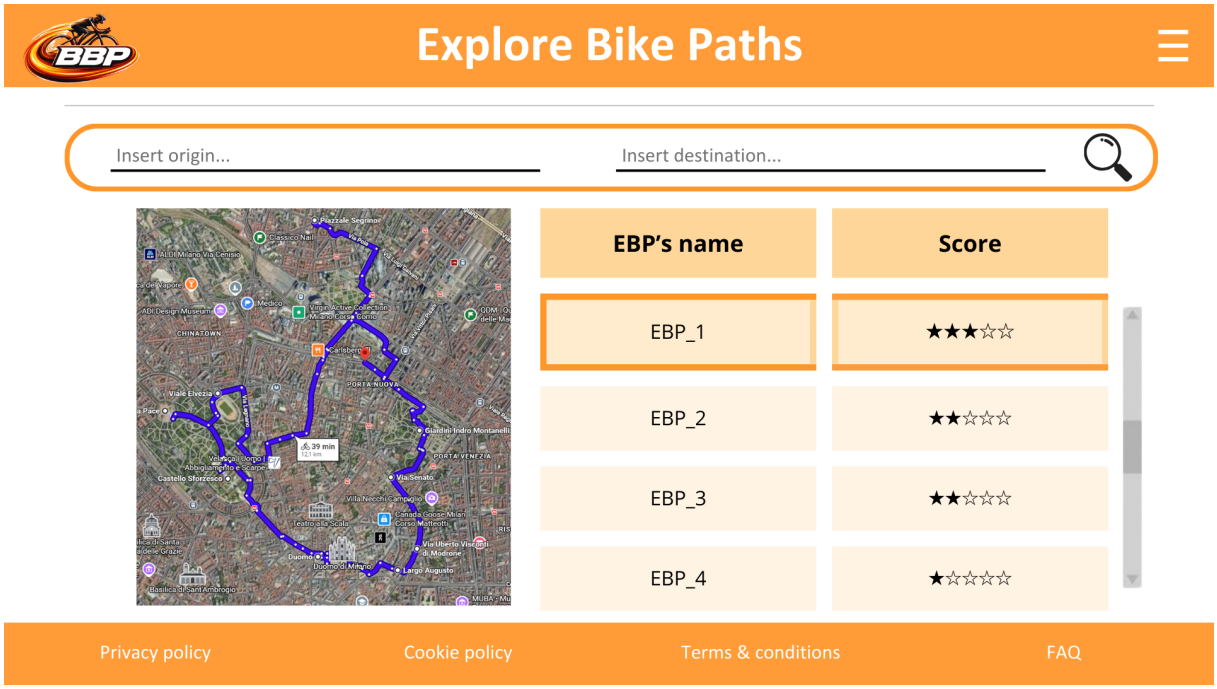


Figura 3.4: Explore Bike Paths view for guest users.

Each row in the EBP list shows the EBP's name and its score. Selecting an EBP from the list updates the path highlighted on the map and tapping again will open its detailed view, which is described in the next subsection. If no EBPs are found or the input is invalid, the view displays an appropriate feedback message instead of the results. If the EGAS is temporarily unavailable, the view still displays the list of EBPs, but the map area is replaced by an error message indicating that the map could not be loaded.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

3.2.4 EBP and BP Details - Guest

From the *Explore Bike Paths* view, the guest can open a detailed view for a selected EBP. This view is implemented as a dedicated view, opened when the user activates the *View Details* button on an EBP below the results list.

The EBP/BP details view presents:

- a summary of the selected EBP, including its total distance, status and score;
- the list of BPs composing the EBP, each with its own status and the identifier of the RU who published it;
- a map focused on the shape of the selected EBP.

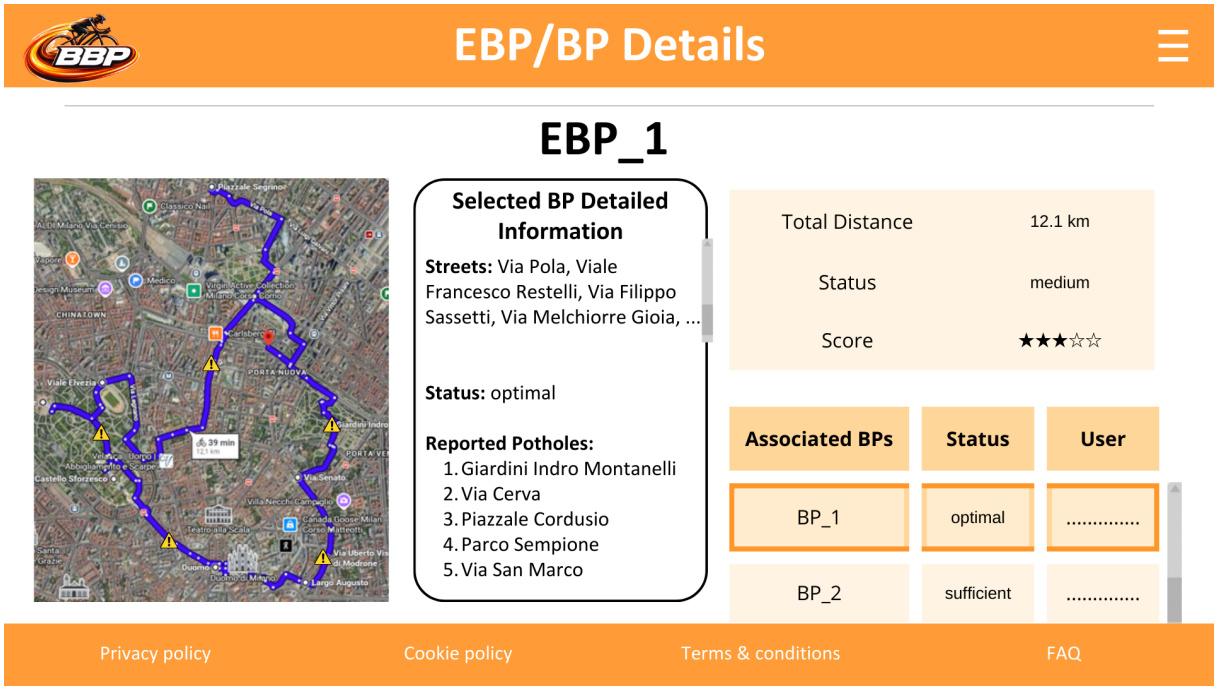


Figura 3.5: Details of a selected EBP and its BPs for a guest user.

When the guest selects one of the BPs in the list, the map highlights that specific BP and, if available, the potholes associated with it. Alongside the map, the interface shows detailed information about the selected BP, such as the streets that compose it, its status description and the positions of reported potholes.

If the EGAS service is unavailable or returns an error while loading the map, the textual information about the EBP and its BPs remain visible and the map area is replaced by a message indicating that the map cannot be displayed.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

3.3 User Flow: Registered User - Trips

RUs have access to all trip-related functionalities of BBP. In addition to exploring published BPs like guests, they can record their own trips, review the collected data and inspect the detailed information associated with each trip.

The trip flow starts when a RU logs in through the access view and is taken to the home view. From there, dedicated entries in the navigation menu allow the user to start a new trip recording or to access the list of previously recorded trips. When recording a trip, the interface shows the current state of the recording and the main metrics; after the recording is stopped, a summary is displayed and the trip is stored so that it can be consulted later from the trips list.

These interactions correspond to the upper part of the right-hand side of the high-level flow diagram in Figure 3.1, where the main steps are: *Access BBP* → *Login* → *Home* → *Record Trip* / *Your Trips* → *Trip Summary* / *Trip Details*.

In the navigation diagram of Figure 3.1, the term *Trip Summary* refers to the final state of the *Record Trip* view after the recording has been stopped and the trip metrics are summarized.

3.3.1 Login and Home - Registered User

RUs access BBP through the same access view described in Section 3.2.1. On this view they enter their *username* and *password* in the login form and submit it using the *Log In* button. In case of invalid credentials, the view shows an error message and keeps the user on the same view; otherwise, the user is authenticated as a RU and the home view is shown.

The home view is the main entry point for trip and BP related functionalities available to RUs. In addition to the common top and bottom bars described in the overview, the central part of the view contains:

- a short welcome area, where the system can display the RU's name and a brief description of the available actions;
- a set of cards providing direct access to the main sections, such as *Record Trip*, *Your Trips*, *Your Bike Paths* and *Explore Bike Paths*.

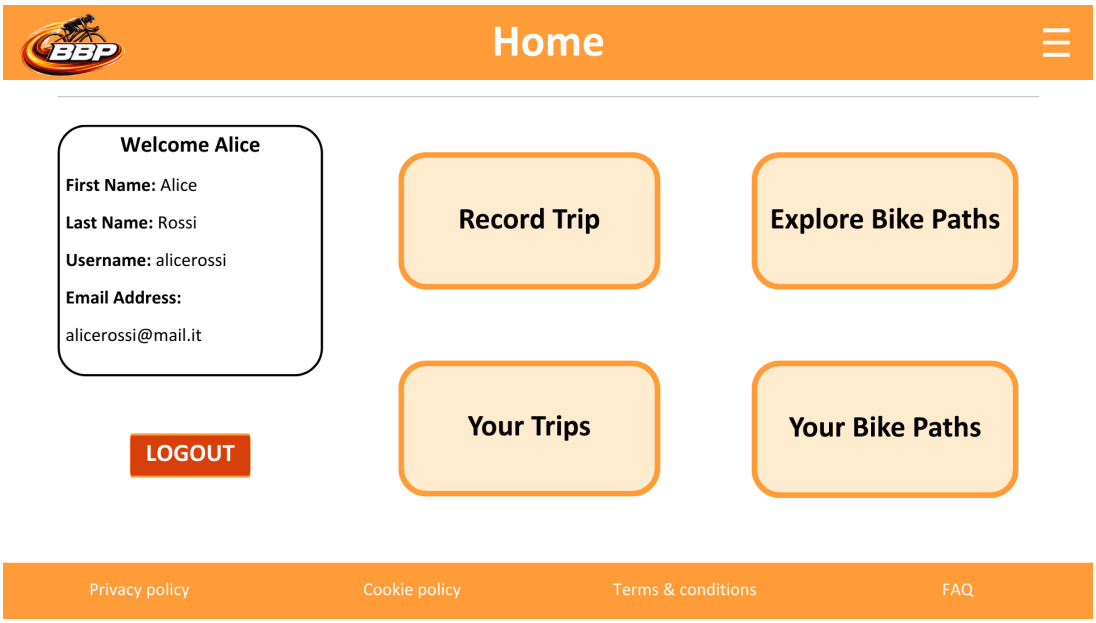


Figura 3.6: Home view for registered users.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

From the home view and any other view, a RU can use the *Log Out* entry in the top bar to terminate the session. When this action is selected, the application clears the session information and returns the user to the access view.

3.3.2 Record Trip view

The *Record Trip* view allows a RU to start and stop the recording of a trip while BBP collects the relevant data from the device sensors and, when available, from the XMS.

The main elements of the view are:

- a text field where the user enters a *Trip Name*, which is required before the trip can be stored;
- a *Start Recording* button that initiates the acquisition of trip data;
- a status panel showing the current state of the recording;
- a set of read-only fields displaying the main trip metrics, such as:
  - elapsed time;
  - distance covered so far;
  - current speed and average speed;
  - meteorological information retrieved from the XMS, when available (if the XMS is unavailable, this field shows *N/A*).
- a *Stop Recording* button that terminates the recording.

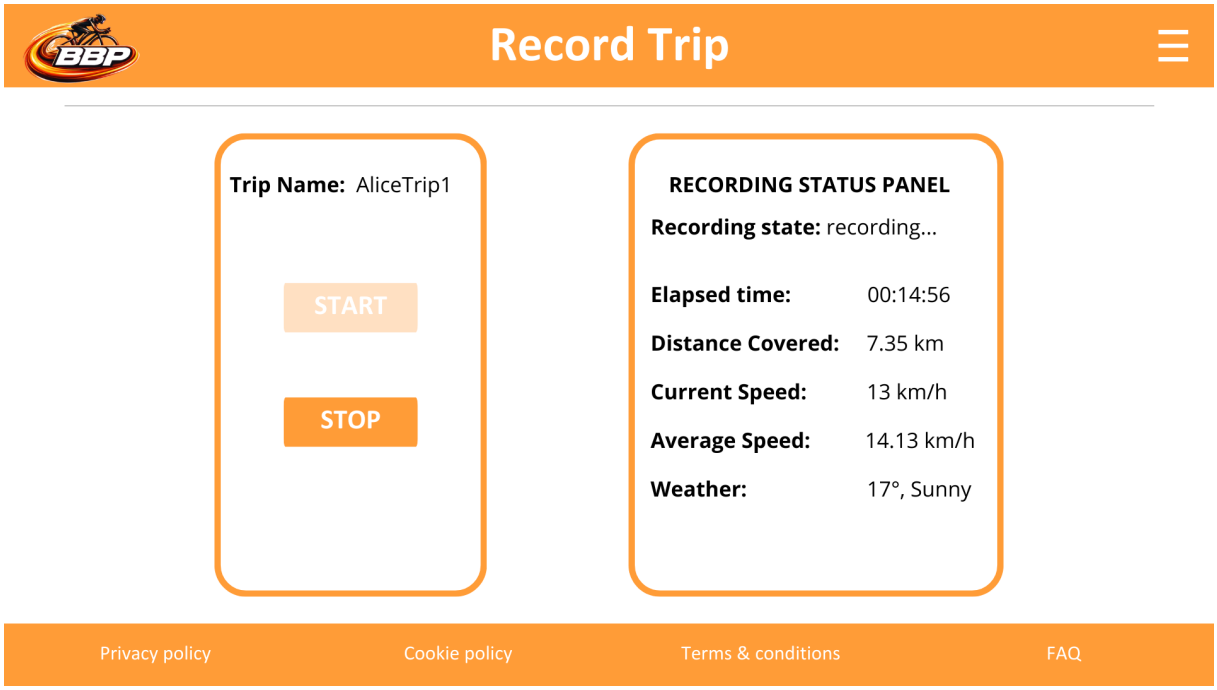


Figura 3.7: Record Trip view for registered users.

While a trip is being recorded, the status panel and the metrics are periodically updated. When the detected speed is zero, the UI reflects that the recording is temporarily paused; if the detected speed exceeds the threshold defined in the requirements (60 km/h), the view shows that the recording has been automatically canceled and that no trip will be stored. If the recording ends normally, that is, when the user presses *Stop Recording*, the view then presents a summary of the recorded trip and asks the user to confirm the Trip Name before the trip is stored.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

3.3.3 Trips List

The *Your Trips* view lets a RU browse the trips that have been recorded and stored by the system.

The main content of the view is a table where each row corresponds to a single trip and shows:

- the *Trip Name*, if provided during recording;
- the *date* and *time* when the trip was recorded;
- the *distance* covered;
- the *average speed* over the whole trip.

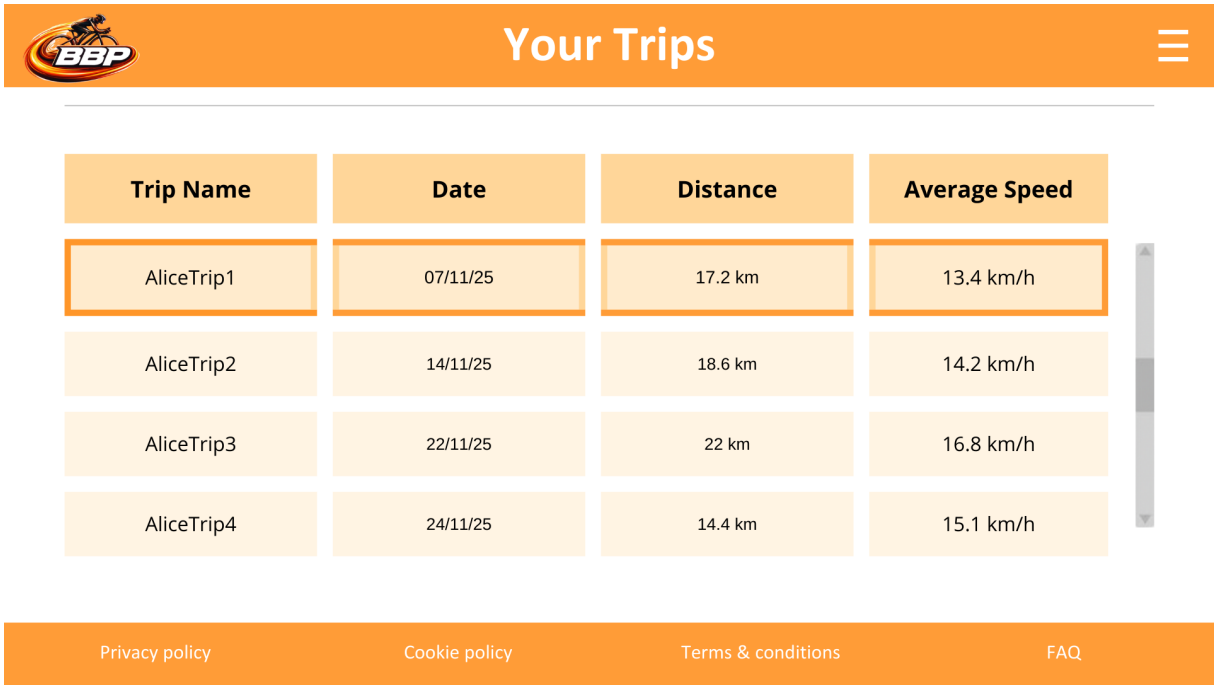


Figura 3.8: "Your Trips" view listing recorded trips.

The rows in the list are selectable: when the user selects a trip, the interface opens the corresponding *Trip Details* view, described in the next subsection. If no trips are available, the view displays a short message instead of the list.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

3.3.4 Trip Details

The *Trip Details* view shows all the information stored for a specific trip selected from the *Your Trips* view.

The view contains:

- the *Trip Name*;
- the *date* and *time* of the trip;
- the *total distance* covered;
- the *average speed*;
- the meteorological information associated with the trip, when retrieved from the XMS (if the XMS was unavailable during recording, this field shows *N/A*).

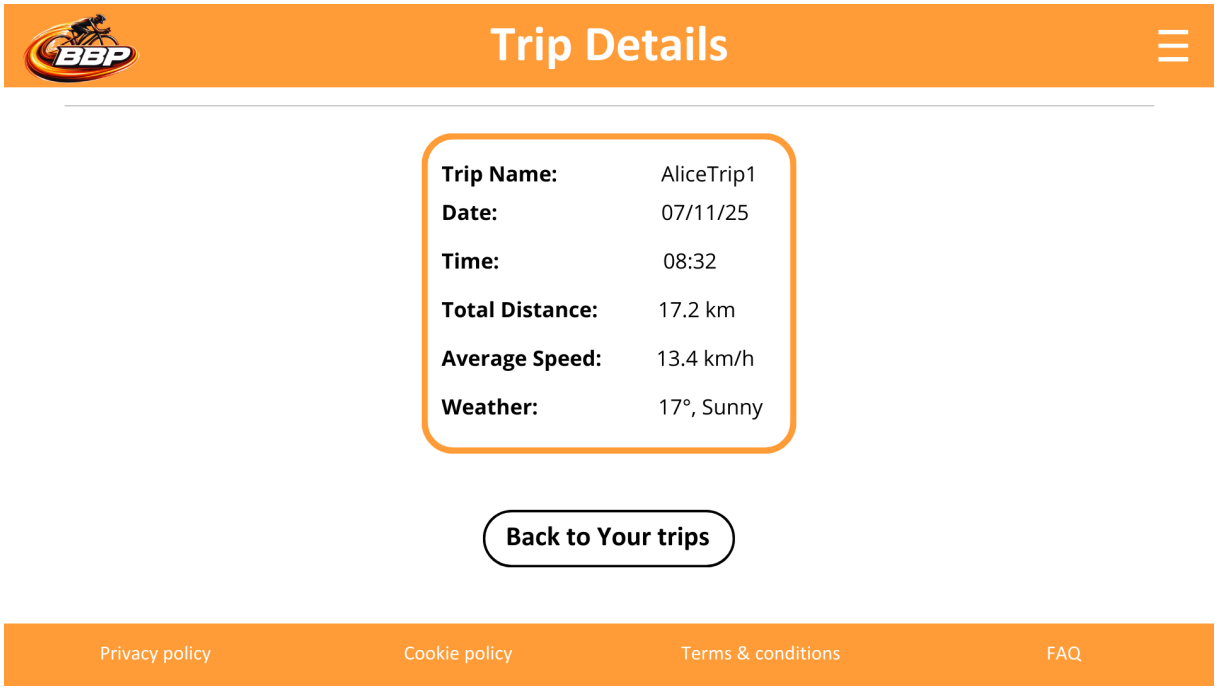


Figura 3.9: Details view for a recorded trip.

The layout includes a compact summary panel that groups the main metrics. A back control (e.g., a *Back to Your Trips* button) returns the user to the trips list without modifying the stored data.

3.4 User Flow: Registered User - BPs

In addition to managing trips, RUs can create, inspect and publish BPs. These functionalities are used to populate the catalog of cycling routes that can later be explored both by RUs and by guest users.

The BP-related flow starts from the home view, where the RU can navigate to the *Your Bike Paths* section. From there, the user can view the list of BPs they have already created, start the creation of a new BP by choosing between a manual insertion mode and an automatic insertion mode based on sensor data and open the detailed view of an existing BP. For each BP, the interface also provides a publication action, which marks it as available for public exploration.

These interactions correspond to the lower part of the right-hand side of the high-level flow diagram in Figure 3.1, where the main steps are: *Access BBP* → *Login* → *Home* → *Your Bike Paths* → *Add New*

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

BP → Choose Insertion Mode → Manual BP Insertion / Automatic BP Recording → Confirmation of Automatically Retrieved Data → BP Details → Publish BP.

When RUs want to explore published BPs, they access the same *Explore Bike Paths* view described for guests in Section 3.2.3, with the only difference that the top bar shows the RU-specific navigation and log out options.

3.4.1 Your Bike Paths

The *Your Bike Paths* view lets a RU manage the BPs they have created in the system. It displays all BPs owned by the current user and provides the entry point for creating new ones.

The main content of the view is a table in which each row corresponds to a single BP and shows:

- the *name* of the BP;
- the current *status* of the BP;
- the *publication state* of the BP (published or not published).

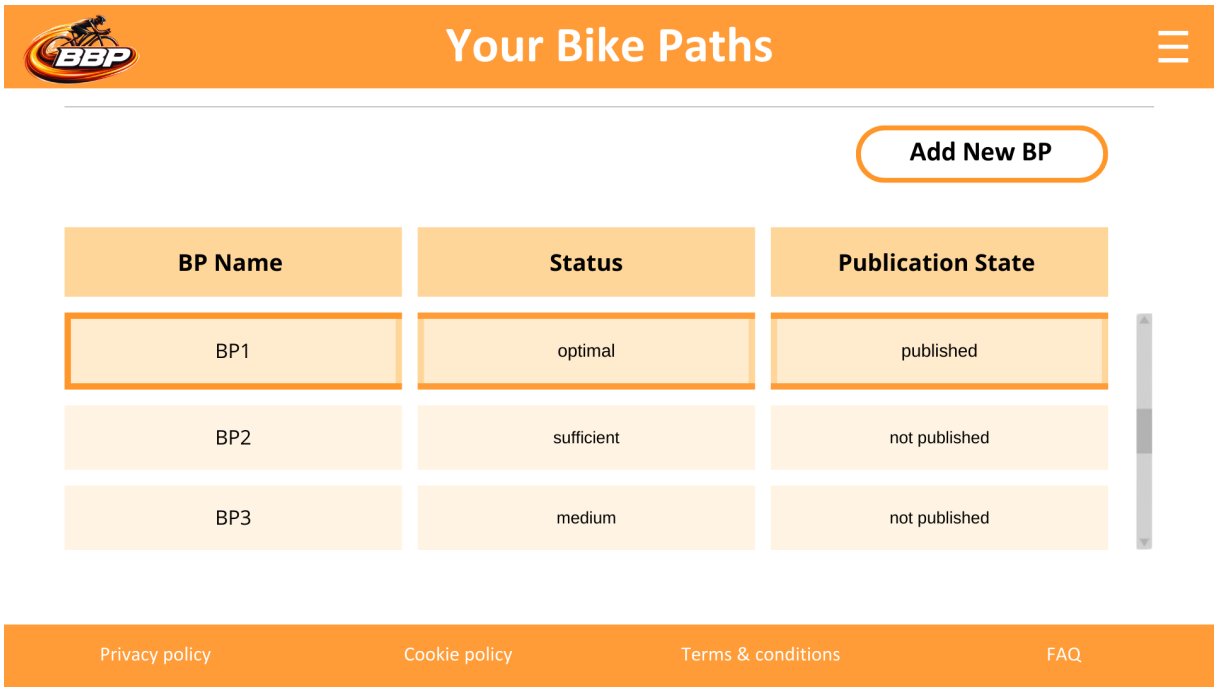


Figura 3.10: "Your Bike Paths" view listing the BPs created by the RU.

Above the list, the view contains an *Add New BP* button. Clicking this button opens the view where the RU selects the insertion mode for the new BP. Rows in the list are selectable: when the user selects a BP clicking on its row, the interface opens the *BP Details* view for that BP. If no BPs are present for the current RU, the view displays an empty state message and still shows the *Add New BP* button.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

3.4.2 Choose Insertion Mode

The *Choose Insertion Mode* view is shown when the RU activates the *Add New BP* button from the *Your Bike Paths* view. Its purpose is to let the user select how the new BP is created.

The view contains a short textual explanation of the two available modes and two distinct controls:

- a *Manual Insertion* button, which leads to the *Manual BP Insertion* view;
- an *Automatic Insertion* button, which leads to the *Automatic BP Recording* view.

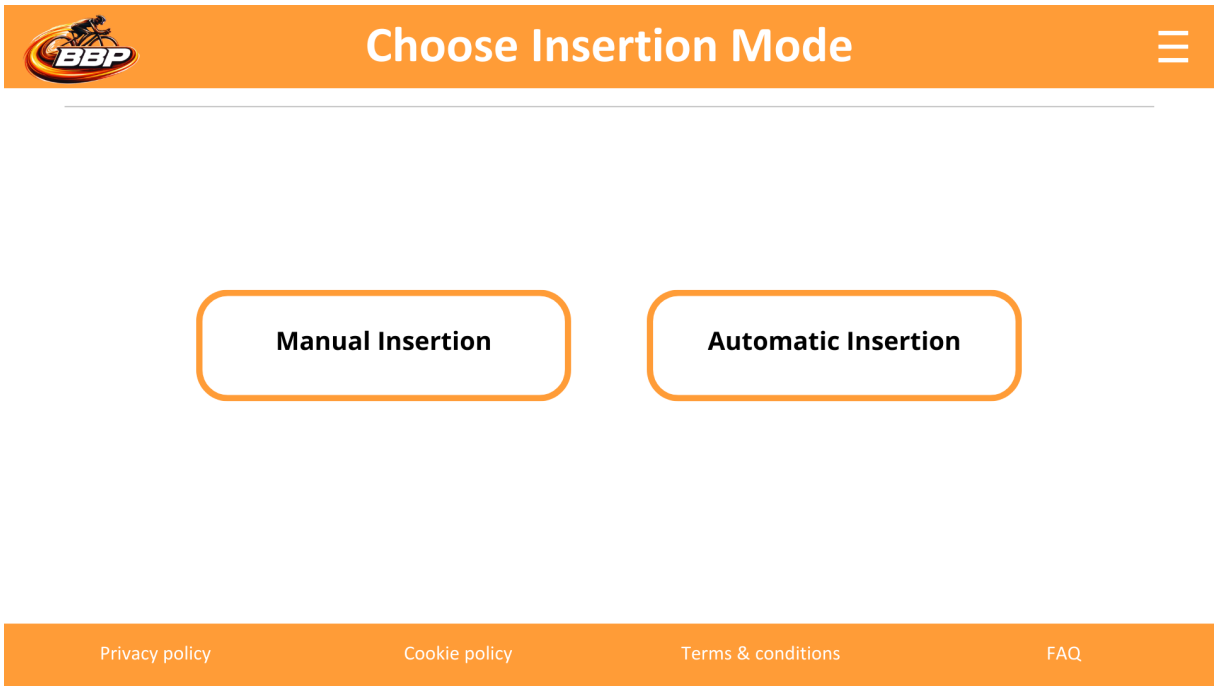


Figura 3.11: View for selecting the insertion mode of a new BP.

The layout presents the two options in a comparable way so that the choice between them is clear. Navigation controls allow the RU to return to the *Your Bike Paths* view without creating a new BP if they decide not to proceed.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

3.4.3 Manual BP Insertion

The *Manual BP Insertion* view is shown when the RU selects the *Manual Insertion* option from the *Choose Insertion Mode* view. It is used to create a new BP by explicitly specifying its streets, status and potholes.

The view contains a form with the following fields:

- a *BP Name* text field, used to assign a name to the new BP;
- a *Streets* section, where the RU specifies the streets that compose the BP. The section includes an input for the street name and controls to add and remove street entries so that the entire path is described;
- a *Status* selector, used to assign the condition of the BP among the values allowed by the system;
- a *Potholes* section, where the RU specifies the potholes detected along the BP. For each pothole the RU indicates the street where it is located and its distance in kilometers from the beginning of the BP.

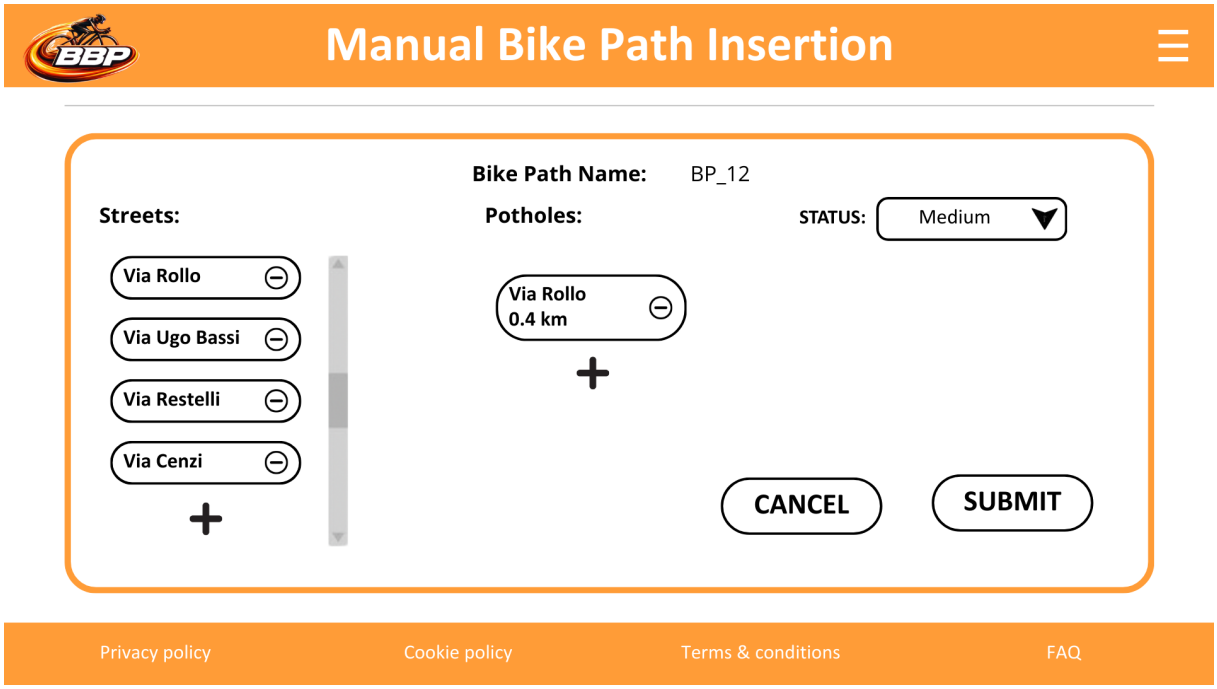


Figura 3.12: Manual insertion view for a new BP.

Below the form, the view includes a *Submit* button and a *Cancel* button. When the RU presses *Submit*, the interface disables the form inputs, shows a loading indication and waits for the validation result from the system. If the validation fails (for example because some streets do not exist, some streets exceeds the allowed speed limit or the EGAS service reports an error), the view shows an error message and re-enables the form so that the RU can correct the data. If the validation succeeds, the view shows a confirmation message and the new BP is added to the list displayed in the *Your Bike Paths* view.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

3.4.4 Automatic BP Recording

The *Automatic BP Recording* view is shown when the RU selects the *Automatic Insertion* option from the *Choose Insertion Mode* view. It is used to create a new BP by letting BBP acquire data automatically while the user is biking.

The view contains:

- a short textual description explaining that the system uses GPS and device sensors to reconstruct the path and detect possible potholes and that the data will have to be confirmed in a later step;
- a *Status* panel indicating the current state of the recording (*Not recording*, *Recording*, *Stopped*);
- a *Start* button that initiates the acquisition of data;
- a *Stop Recording* button that terminates the acquisition;
- a set of read-only fields showing:
  - elapsed time since the start of the recording;
  - approximate distance covered;
  - current speed as detected from the sensors;
  - the number of potential potholes detected along the path.

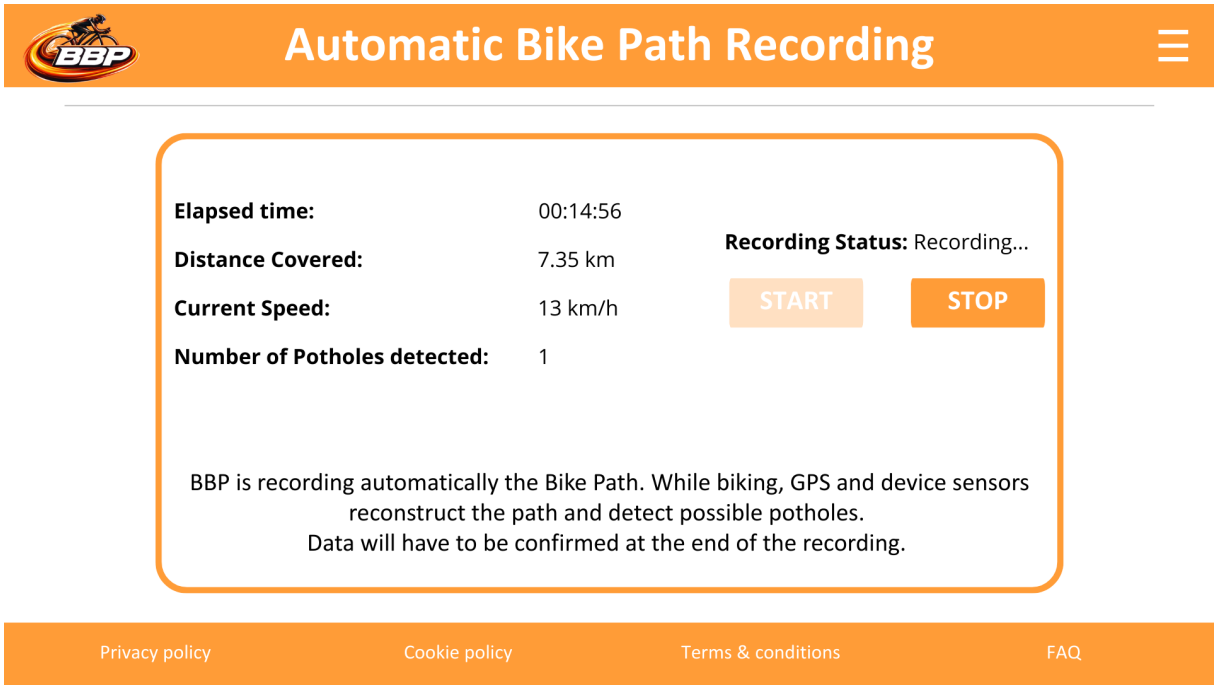


Figura 3.13: Automatic recording view for a new BP.

When the RU presses *Start*, the view switches to the *Recording* state and the status panel and metrics are updated as new data arrive; the *Stop Recording* button becomes active. If the detected speed is zero, the status indicates that the user is not moving. If the detected speed exceeds the threshold defined in the requirements (60 km/h), the view shows that the recording has been automatically canceled and that no BP will be created from this recording. When the RU presses *Stop Recording* and the recording ends normally, the acquisition for the new BP is completed and the interface proceeds to the confirmation view described in the next subsection.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

3.4.5 Confirmation of Automatically Retrieved Data

The *Confirmation of Automatically Retrieved Data* view is shown immediately after the automatic recording of a new BP ends. Its purpose is to let the RU inspect and adjust the data collected by the system before the BP is stored.

The view contains:

- a *BP Name* text field, used to assign a name to the new BP;
- a *Reconstructed Path* section, where the streets composing the BP are shown as a textual list in the order in which they are traversed;
- a *Potholes* section, which lists all potential potholes detected by the system during the recording. For each pothole the view shows the street where it is located and its distance in kilometers from the beginning of the BP;
- a *Status* selector, used to assign the condition of the BP among the values allowed by the system.
- a *Score* field showing the score automatically computed by BBP, which the RU can confirm or adjust if needed.

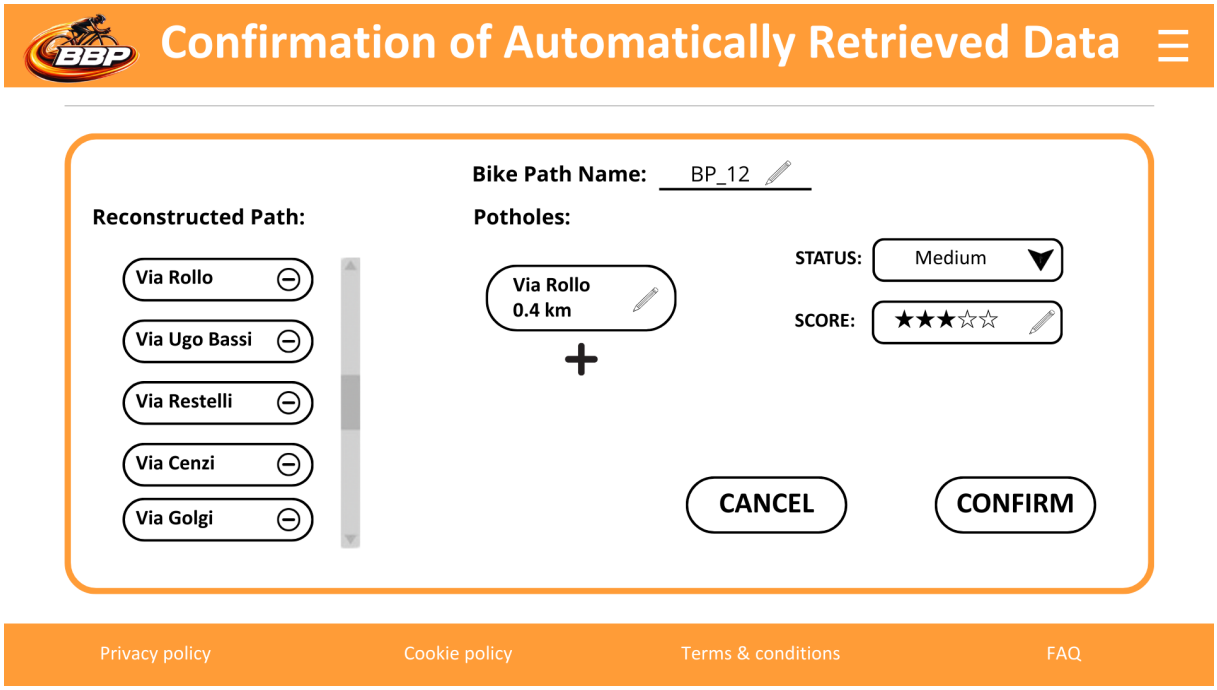


Figura 3.14: Confirmation view for data collected during automatic BP recording.

In the *Potholes* section, the RU edits the automatically retrieved data. For each pothole, the RU can confirm that it is correct or delete it if it is a false positive. The view also provides controls to add new potholes that were not detected automatically, specifying their position along the BP. The reconstructed BP section allows the RU to verify that the streets and shape of the BP are correct.

At the bottom of the view there is a *Confirm* button and a *Cancel* button. When the RU presses *Confirm*, the system saves the BP using the streets, potholes and status currently shown in the view and adds it to the list in the *Your Bike Paths* view. If the RU presses *Cancel*, the automatic insertion is discarded and no new BP is created.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

3.4.6 BP Details and Publish

The *BP Details* view is shown when the RU selects a BP from the *Your Bike Paths* view. It is used to inspect all the information stored for that BP and to publish it if it is not yet published.

The view contains:

- the *name* of the BP;
- the *status* assigned to the BP;
- a *Streets* section that lists the streets composing the BP in order;
- a *Potholes* section that lists the potholes associated with the BP, including their positions along the BP;
- a *Publication state* field that indicates whether the BP is published or not.
- a *Score* field indicating the BP's score used for EBP ranking.

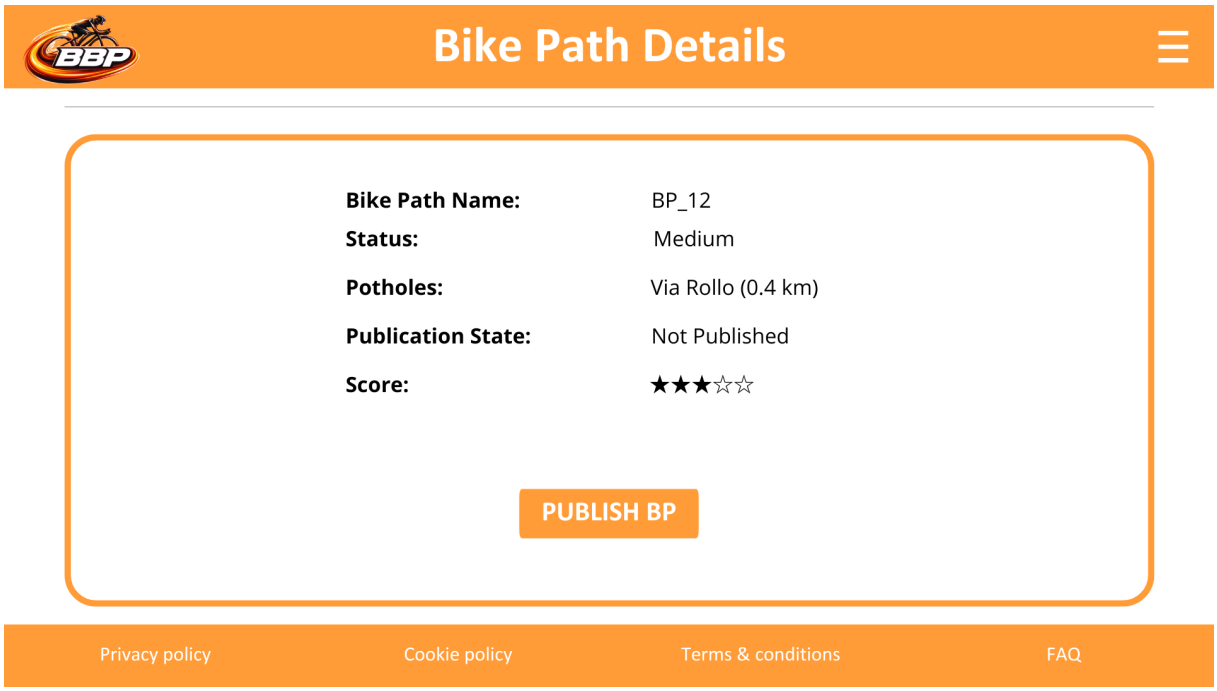


Figura 3.15: Details and publication view for a BP.

At the bottom of the view there is a *Publish BP* button when the BP is not yet published. Activating this button sends the publication request to the system and disables the button while the operation is processed. If the publication succeeds, the view updates the publication state to *published* and the BP is shown as published in the *Your Bike Paths* view. If the publication fails, an error message is displayed and the button becomes available again so that the RU can retry after correcting possible issues. A navigation control allows the user to return to the *Your Bike Paths* view without changing the publication state.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

4 Requirements Traceability

Requirement	Components
R1	View, Application Manager, API Manager, Users Manager, DBMS
R2	View, Application Manager, API Manager, Users Manager, DBMS, Local DB
R3	View, Application Manager, API Manager, Users Manager, DBMS, Local DB
R4	View, Application Manager
R5	View, Trip Manager, API Manager, Trip Saver, DBMS, Local DB, XMS API, GPS
R6	Trip Manager, GPS
R7	Trip Manager
R8	Trip Manager
R9	Trip Manager, API Manager, Trip Saver, XMS API
R10	View, Trip Manager, Local DB
R11	View, BPs Manager, API Manager, BP Saver, EGAS API, DBMS, Local DB
R12	View, BPs Manager, API Manager, BP Saver, EGAS API
R13	View, BPs Manager
R14	View, BPs Manager, GPS, Sensors, API Manager, BP Saver, EGAS API, DBMS, Local DB
R15	BPs Manager, GPS, API Manager, BP Saver, EGAS API
R16	BPs Manager, Sensors
R17	View, BPs Manager
R18	View, BPs Manager
R19	View, BPs Manager
R20	View, BPs Manager
R21	View, Application Manager, API Manager, EBPs Manager, DBMS
R22	View, Application Manager
R23	View, Application Manager, API Manager, Users Manager, DBMS, EGAS API
R24	View, Application Manager, API Manager, Users Manager, DBMS, EGAS API
R25	View, Application Manager, API Manager, Users Manager, DBMS
R26	View, Application Manager, API Manager, Users Manager, DBMS, EGAS API
R27	View, BPs Manager, Local DB

Table 1: Requirements Traceability

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

5 Implementation, Integration Test Plan

5.1 Overview

In this chapter, we discuss the implementation of the BBP platform, the integration process and the testing strategy adopted to ensure the quality of the software. The primary goal of testing is to identify and fix most defects or issues introduced during development. The integration process is described in Section 5.4, while the implementation strategies are presented in Section 5.2.

5.2 Implementation Plan

The implementation follows a bottom-up approach, starting from the lowest-level modules and progressively integrating them into higher-level components, consistently with the adopted 3-tier fat-client architecture.

1. **Contracts and data model.** Define the shared domain entities and the REST API contract (JSON formats, endpoints, error codes), ensuring consistency across client and server.
2. **Persistence foundations.** Implement the server-side DB schema and access layer and the client-side local DB with its data-access utilities, to support both online and offline workflows.
3. **External services adapters.** Implement the EGAS and XMS integration layers, together with stub versions to enable development and testing independently from third-party availability.
4. **Server-side business components.** Implement Users Manager, Trip Saver, BP Saver and EBPs Manager on top of the DBMS and external adapters.
5. **API routing and deployment alignment.** Implement the API Manager logic coherently with the deployment choice, separating the request flows handled by Application Servers and Synchronization Servers.
6. **Client-side business components.** Implement Application Manager, Trip Manager and Bike Paths Manager, including the real-time acquisition loops required for GPS and sensors.
7. **Synchronization.** Implement the Synchronizer component, enforcing the alignment policy between local DB and server DB and integrating it with connectivity state changes.
8. **Presentation layer.** Implement the View and its handlers/listeners, integrating all user flows on top of the stabilized client-side components.

5.3 Features Identification

This section identifies the main functional features of BBP. These features will be used as reference increments in the integration strategy described in Section 5.4.

[F1] Account Access and Session Management This feature covers account registration, RU login and logout and access as guest. It also includes the management of authenticated sessions and related access control for restricted functionalities.

[F2] Trip Recording This feature allows a RU to record a trip by acquiring GPS data, computing the required trip statistics and enriching the trip with meteorological information when available. The trip recording workflow manages start, pause and stop conditions and stores the collected information for later use.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

[F3] Trip Navigation This feature allows a RU to view previously recorded trips and their related information. It includes the visualization of trip details and the selection of a specific trip for further actions.

[F4] Manual BP Insertion and Validation This feature allows a RU to manually insert a BP, validate it through the EGAS, report potholes and provide the required BP information. The inserted BP is then stored and made available for further management by the RU.

[F5] Automatic BP Insertion and Confirmation This feature allows a RU to insert a BP automatically by collecting GPS and sensor data, reconstructing the path, inferring potential potholes and confirming or correcting the detected data before saving. The RU can review the generated BP and adjust its information before it becomes persistent.

[F6] BPs Navigation and Publication This feature allows a RU to view previously inserted BPs and to publish a BP, making it available for exploration. If an EBP with the same streets already exists, the published BP is associated to it, otherwise a new EBP is created.

[F7] Explorable BPs Search and Visualization This feature allows any user to search EBPs between an origin and a destination, obtain results ordered by score, visualize a selected EBP on the map and inspect its detailed information. The user can browse alternative results and switch the displayed EBP accordingly.

5.4 Integration Strategy

This section describes the integration strategy adopted to progressively assemble the BBP system starting from the components defined in the design. The integration is carried out gradually in order to limit complexity, validate interfaces as soon as they are introduced and localize faults when unexpected behaviors occur. The adopted approach is consistent with the bottom-up implementation plan: lower-level modules and services are integrated first and then used as a stable baseline to integrate higher-level components.

5.4.1 Integration Approach

The integration of BBP follows an incremental bottom-up strategy, consistently with the implementation plan and with the adopted 3-tier fat-client architecture. Each integration increment is validated through dedicated integration tests focusing on the interactions among the newly integrated components and on the correctness of the exchanged data. When higher layers are not yet available, suitable drivers are employed to exercise the integrated subset.

External dependencies are handled in two phases: first, stubbed versions of EGAS and XMS are used to develop and validate the internal interactions; then, the stubs are replaced with the real services to verify the correct behavior of the system under realistic conditions, including error cases and service unavailability.

The integration process explicitly accounts for the client-server communication through the REST API and for the deployment choice in which the API Manager is split across Application Servers and Synchronization Servers. After each integration increment, a regression set of integration tests is executed to ensure that previously integrated interactions still behave as expected.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

5.4.2 Integration Steps

In the following, each step specifies the components integrated in the increment, the adopted drivers and stubs, when needed and the main integration tests executed to validate the newly introduced interactions.

Step 1 - Data Model and Persistence Foundations This step integrates the persistence foundations on both tiers by setting up the shared data model and the two data stores used by BBP. In particular, the server-side DBMS schema and access utilities are implemented together with the client-side Local DB schema and its data-access utilities, so that core entities can be stored and retrieved consistently in both online and offline scenarios.

- **Integrated components:** DBMS, Local DB (including their schemas and data-access utilities), shared domain entities.
- **Stubs/Drivers:** Persistence driver(s) to exercise read/write operations without relying on higher-level business components or the UI.
- **Integration tests:**
  - creation, retrieval, update and deletion of the main stored entities on the **DBMS**;
  - creation, retrieval, update and deletion of the main stored entities on the **Local DB**;
  - consistency checks on identifiers and stored attributes needed for later client-server exchanges;
  - validation of referential integrity among stored entities.

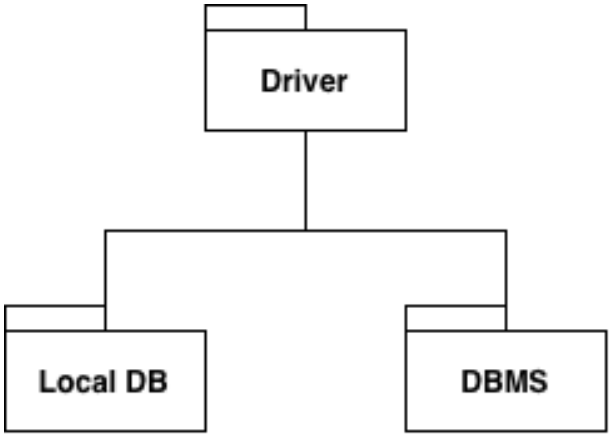


Figura 5.1: Integration Step 1: Local DB and DBMS Integration.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

Step 2 - Users Management Integration This step integrates the server-side user management logic on top of the DBMS, enabling the basic operations required to manage accounts and authenticated access. The goal is to validate the interaction between the *Users Manager* and the persistence layer before introducing the API layer and the client-side components.

- **Integrated components:** Users Manager, DBMS.
- **Stubs/Drivers:** Users Manager driver to invoke the Users Manager operations without relying on the API layer or the UI.
- **Integration tests:**
  - user registration and persistence of credentials and user data in the DBMS;
  - authentication with valid credentials and rejection of invalid credentials;
  - logout/session invalidation;
  - validation of error handling for duplicated accounts and missing users.

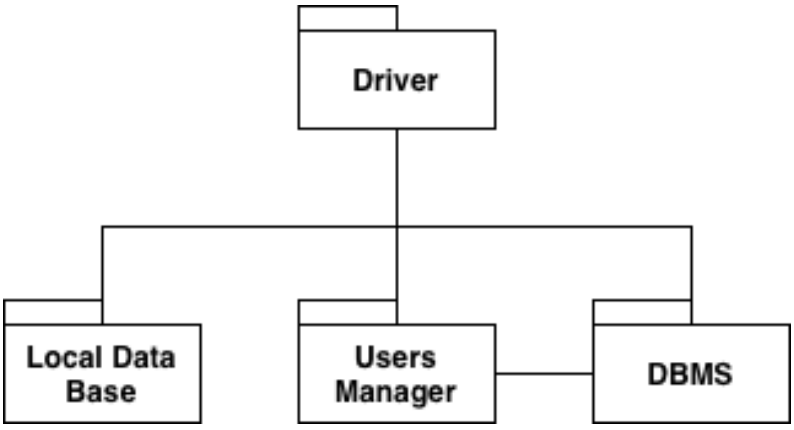


Figura 5.2: Integration Step 2: Users Manager and DBMS Integration.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

Step 3 - API Manager and Users Management Integration This step integrates the API layer on the Application Servers with the already validated user management logic, enabling the first end-to-end server workflow through the REST interfaces. The goal is to verify that the API Manager correctly routes requests related to account access and session management to the Users Manager and that the expected responses and error cases are handled consistently.

- **Integrated components:** API Manager (Application Servers), Users Manager, DBMS.
- **Stubs/Drivers:** API driver to simulate client-side REST calls to the API Manager.
- **Integration tests:**
  - registration requests forwarded by the API Manager and correct persistence through the Users Manager;
  - login requests and correct handling of valid/invalid credentials;
  - logout requests and session invalidation according to the adopted session mechanism;
  - validation of error responses for duplicated accounts and missing users;
  - validation of returned formats and status codes defined by the API contract.

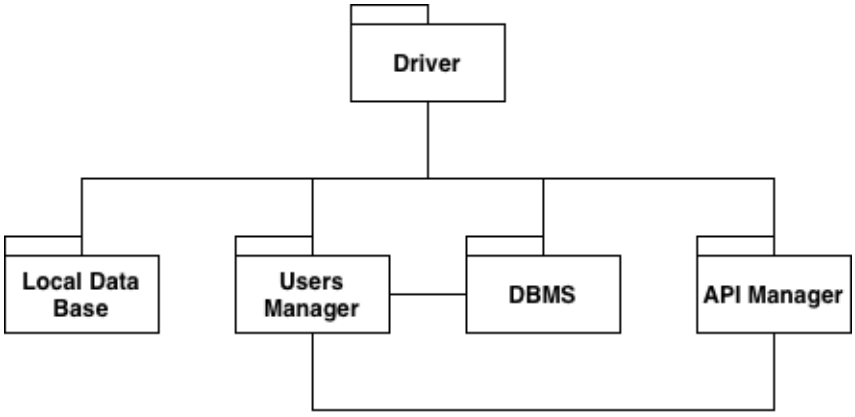


Figura 5.3: Integration Step 3: API Manager and Users Manager Integration.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

Step 4 - Trip Saver Integration (with XMS Stub) This step integrates the server-side trip management logic by introducing the Trip Saver component on top of the DBMS and connecting it to the API Manager deployed on the Application Servers. Since trip information can be enriched with meteorological data, the interaction with XMS is initially validated through a stubbed version of the service, which will later be replaced by the real API.

- **Integrated components:** API Manager (Application Servers), Trip Saver, DBMS, XMS API (stub).
- **Stubs/Drivers:** API driver to simulate client-side REST calls related to trips and XMS stub to emulate meteorological responses and failures.
- **Integration tests:**
  - correct routing of trip-related requests from the API Manager to the Trip Saver;
  - persistence of trip data through the Trip Saver into the DBMS;
  - enrichment of trip data through calls to the XMS stub and correct handling of missing or degraded responses;
  - validation of returned formats and error handling as defined by the API contract.

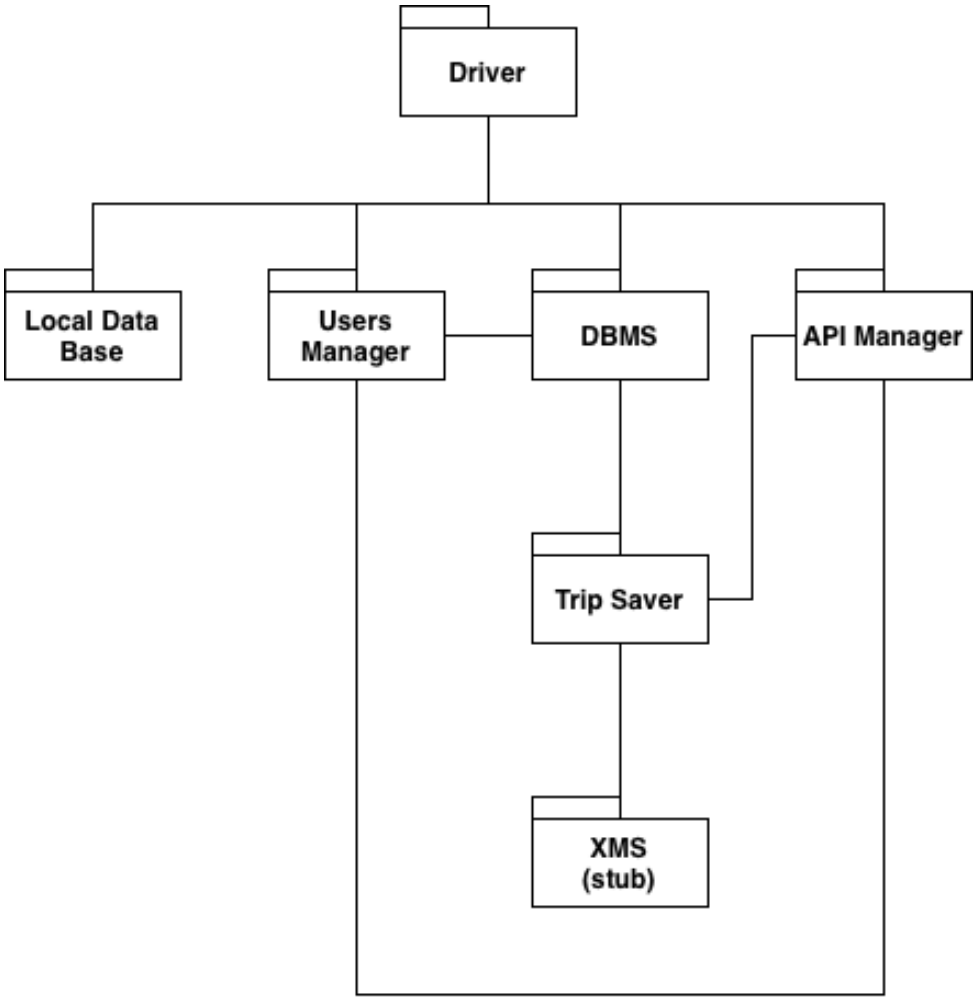


Figura 5.4: Integration Step 4: API Manager and Trip Saver Integration.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

Step 5 - BP Saver Integration (with EGAS Stub) This step integrates the server-side management of BPs by introducing the BP Saver component. The BP Saver is connected to the API Manager on the Application Servers and persists the BP-related data on the DBMS. Since BP insertion and validation rely on the EGAS service, the interaction with EGAS is first validated through a stubbed version of the service, which will later be replaced by the real API.

- **Integrated components:** API Manager (Application Servers), BP Saver, DBMS, EGAS (stub).
- **Stubs/Drivers:** API driver to simulate client-side REST calls related to BP insertion and management; EGAS stub to emulate validation/conversion responses and failures.
- **Integration tests:**
  - correct routing of BP-related requests from the API Manager to the BP Saver;
  - persistence of manually inserted BP data through the BP Saver into the DBMS;
  - validation/conversion requests to the EGAS stub and correct handling of negative results or unavailability;
  - persistence of automatically generated BP data (when submitted by the client) and storage of associated metadata (e.g. potholes and status);
  - validation of returned formats and error handling as defined by the API contract.

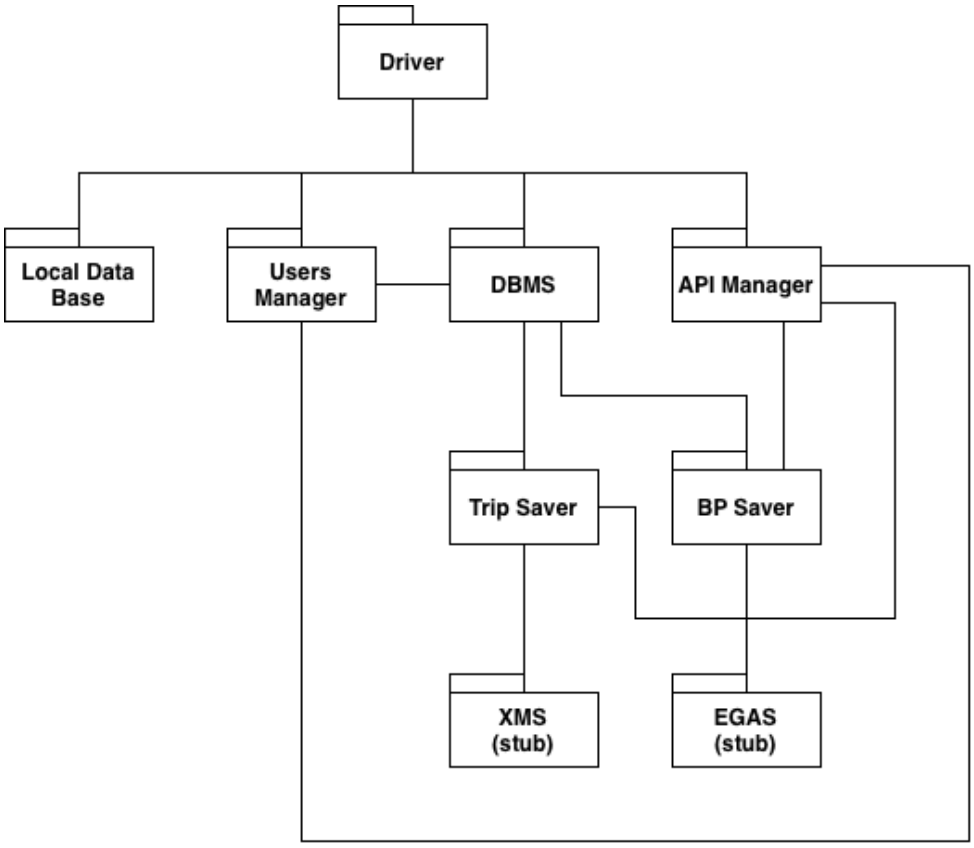


Figura 5.5: Integration Step 5: BP Saver and DBMS Integration with EGAS (stub).

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

Step 6 - EBPs Manager Integration (with EGAS Stub) This step integrates the server-side exploration and publication logic by introducing the EBPs Manager. The EBPs Manager is connected to the API Manager and relies on the DBMS to retrieve and update the information required to manage EBPs. Since exploration requires the interaction with the EGAS service to obtain map data, this interaction is initially validated through the EGAS stub, which will be replaced by the real service in the final integration.

- **Integrated components:** API Manager (Application Servers), EBPs Manager, DBMS, EGAS (stub).
- **Stubs/Drivers:** API driver to simulate REST calls related to publication and exploration; EGAS stub to emulate map retrieval responses and failures.
- **Integration tests:**
  - correct routing of publication and exploration requests from the API Manager to the EBPs Manager;
  - publication workflow: association of a published BP to an existing EBP when applicable, otherwise creation of a new EBP, with correct persistence on the DBMS;
  - exploration workflow: retrieval of EBPs between an origin and a destination ordered by score;
  - map retrieval for a selected EBP through the EGAS stub and correct handling of negative results or service unavailability;
  - validation of returned formats and error handling as defined by the API contract.

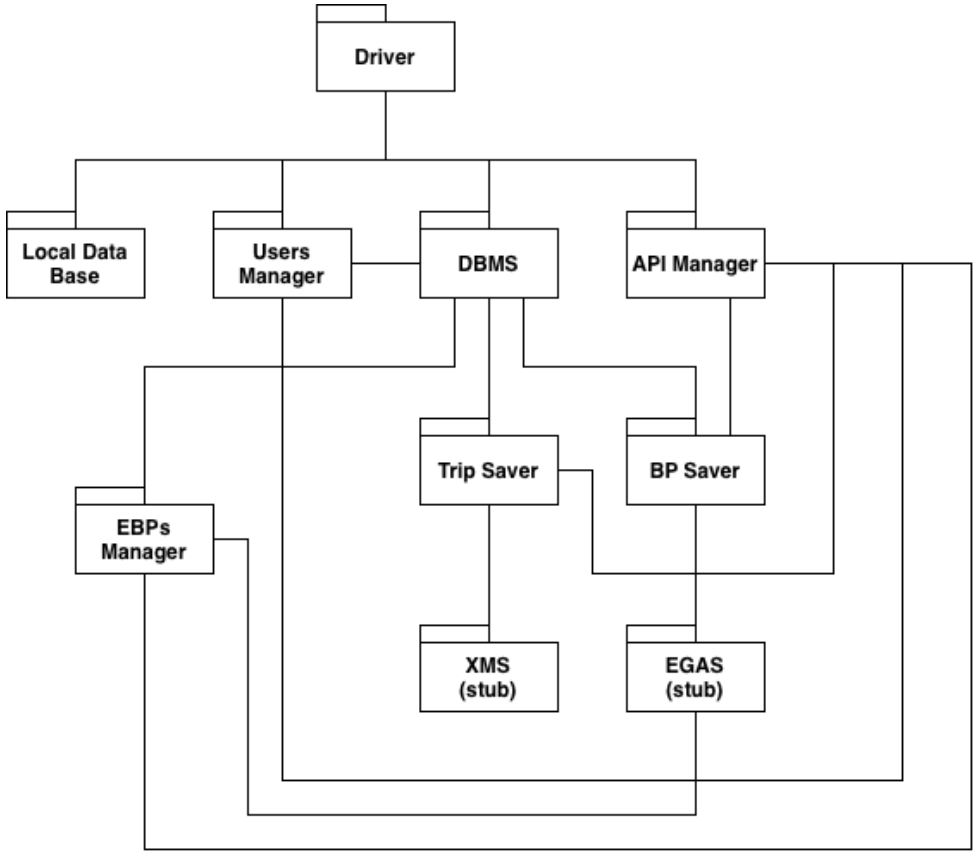


Figura 5.6: Integration Step 6: EBPs Manager and DBMS Integration with EGAS (stub).

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

Step 7 - Client-side Components, Synchronization and End-to-End Integration This step completes the integration by introducing the client-side business components and the presentation layer, enabling end-to-end execution of BBP workflows. The View is integrated with the Application Manager, Trip Manager and Bike Paths Manager, while the Synchronizer is connected to the Local DB and to the server-side APIs to handle alignment between local and remote data when connectivity changes. In this increment, the external services are exercised through their real interfaces to validate the overall behavior of the system under realistic conditions.

- **Integrated components:** View, Application Manager, Trip Manager, Bike Paths Manager, Synchronizer, Local DB, API Manager (Application Servers), API Manager (Synchronization Servers), Users Manager, Trip Saver, BP Saver, EBPs Manager, DBMS, EGAS API, XMS API.
- **Stubs/Drivers:** end-to-end driver simulating user interactions and connectivity changes.
- **Integration tests:**
  - account access and session handling through the View and Application Manager, including persistence and reuse of session information on the Local DB;
  - trip recording workflow through the Trip Manager, with local persistence and upload to the server when online, including meteorological enrichment through the XMS API;
  - manual and automatic BP insertion workflow through the Bike Paths Manager, with local persistence and validation/conversion through the EGAS API when required;
  - publication and exploration workflow, including retrieval of EBPs, map-related data through the EGAS API and visualization of details at the View level;
  - synchronization workflows: alignment after login and after connectivity recovery, verifying correct propagation of locally stored trips and BPs and correct retrieval of server-side data;
  - regression execution of the previously validated server-side interactions.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

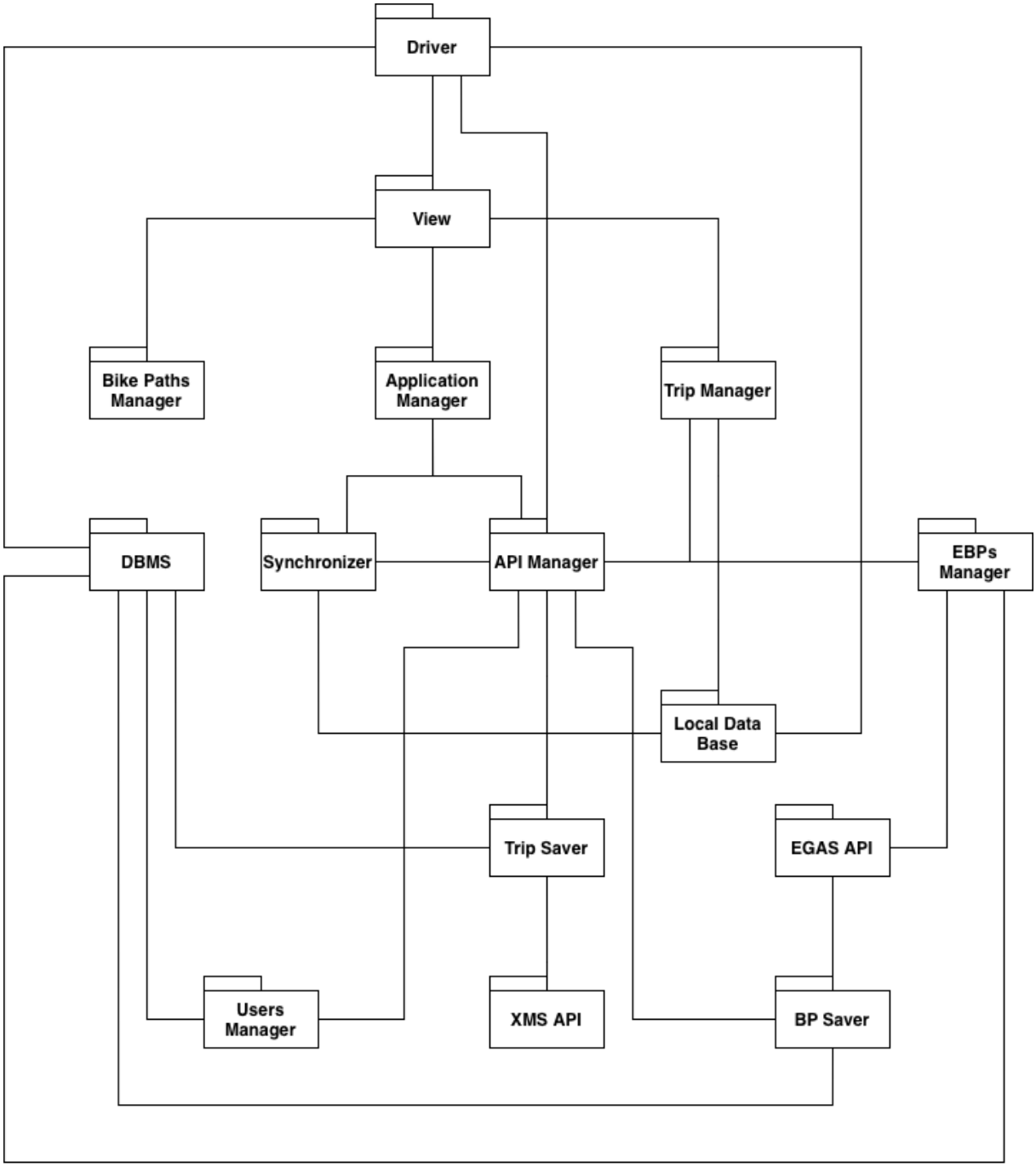


Figura 5.7: Integration Step 7: End-to-End Integration with Client-side Components and Synchronizer.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

5.5 System Testing Strategy

After the integration activities described in Section 5.4 are completed, BBP is tested as a whole system. System testing is conducted mainly through a black-box approach, evaluating the platform from the users' perspective and focusing on observable behaviors and outcomes, rather than on internal implementation details.

The system testing strategy is organized into the following test categories.

1. **Functional Testing.** The objective is to verify that the end-to-end workflows supported by the platform behave correctly and consistently. Functional system tests cover the complete user journeys, including registration/login/logout, trip recording and navigation, BP insertion (manual and automatic with confirmation), publication and exploration of EBPs on the map. Particular attention is given to the correctness of persisted data, to the coherence between the local and server states and to the proper handling of invalid inputs and boundary cases.

2. **Performance and Scalability Testing.** The objective is to validate that BBP remains responsive under expected and peak workloads. Performance tests evaluate response time, resource usage and throughput for the most critical operations (e.g., sustained EBP search requests/sec, map visualization latency and ingestion rate during synchronization). Scalability tests evaluate the behavior of the backend under concurrent usage, including high numbers of guest users browsing EBPs and registered users recording trips and inserting BPs, as well as the ability to handle temporary load peaks without significant degradation.

3. **Security Testing.** The objective is to verify that BBP protects sensitive data (credentials, GPS traces and user-generated information) and enforces access control correctly. Security tests focus on authentication and session handling, authorization to restricted functionalities, correct isolation of user data and secure communication channels. The tests also verify that integrations with external services do not expose sensitive information and that the system behaves safely under malformed requests.

4. **Failure and Resilience Testing.** The objective is to validate that BBP behaves correctly when faults occur. Tests simulate network instability and temporary unavailability of external services (XMS and EGAS) to confirm that the system degrades gracefully, continues operating when possible (e.g., omitting optional information) and preserves data consistency. Additional tests verify recovery after faults (e.g., reconnection and late synchronization) and ensure that data is not partially stored or corrupted. The same scenarios are also executed in long-running sessions to detect memory/resource leaks and progressive degradation.

5. **User Acceptance Testing.** A final acceptance session executes the main end-to-end user journeys to validate usability and correctness from a user perspective.

5.6 Additional Specifications on Testing

System tests are executed in an environment consistent with the target deployment (mobile client, Application Servers, Synchronization Servers and DBMS). For tests involving external dependencies, EGAS and XMS are first replaced with stubs to enable deterministic executions and then the same test suites are repeated against the real services to validate integration and fault-handling behavior. Regression tests are executed after each integration increment to ensure that previously validated functionalities are not broken.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

6 Effort Spent

Here is a table showing the effort spent by the two members of the group:

Members	Chapter 1	Chapter 2	Chapter 3	Chapter 4	Chapter 5
Costa Federico	1h	5h	20h	1h	5h
De Simone Emanuele	1h	20h	5h	1h	2h

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

7 References

7.1 References

- The Requirement Engineering and Design Project specification document A.Y. 2025-2026.
- Slides of the Software Engineering 2 course

7.2 Used Tools

- **draw.io** for UML diagrams
- **Overleaf** to write the document
- **GitHub** for versioning
- **Alloy** for formal analysis
- **ChatGPT/Gemini** used to learn LaTeX syntax and to correct some English sentences. The produced outputs were manually reviewed and adapted to the BBP terminology and then cross-checked against the course slides and the project documents. No AI-generated content was included without validation.

  An example of a prompt and its output is reported below:

  Input In recent years, cycling has become an increasingly popular and sustainable mean of transport.

  However, cyclists often face difficulties to find accurate and updated information about BPs. Existing data are usually scattered, incomplete, or not maintained, making it hard to evaluate the quality, safety, and usability of available routes. This lack of trustable information affects both occasional and regular cyclists, who can encounter poorly maintained paths, obstacles, or unsuitable routes without notice.

  Il testo sopra è corretto?

  Output Il testo è molto chiaro e comprensibile, ma ci sono alcuni piccoli errori grammaticali e lessicali che lo rendono poco naturale ("unnatural") in inglese.

  Ecco un'analisi dettagliata delle correzioni necessarie per renderlo perfetto.

  Principali correzioni "Mean of transport" → "Means of transport"

  In inglese, il sostantivo per "mezzo" (di trasporto) è sempre means, anche al singolare. "Mean" significa "cattivo" o "media matematica".

  Corretto: "...a sustainable means of transport."

  "Face difficulties to find" → "Face difficulties (in) finding"

  L'espressione "difficulties to find" non è idiomatica. È meglio usare "face difficulties in finding", "have difficulty finding", oppure "struggle to find".

  L'acronimo "BPs"

  A meno che tu non abbia definito "BPs" (Bike Paths) in precedenza nel testo, non è un acronimo standard riconosciuto universalmente. È meglio scrivere "bike paths" o "cycling routes" per chiarezza.

  "Trustable" → "Reliable"

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved

"Trustable" esiste, ma è raramente usato in contesti formali o tecnici. Reliable (affidabile) o trustworthy (degno di fiducia) suonano molto più professionali.

"Data are" vs "Data is"

Hai scritto "Existing data are". Questo è tecnicamente corretto (specialmente in ambito accademico scientifico), ma nell'inglese moderno "data" viene spesso trattato come singolare ("data is"). Puoi lasciarlo così se il contesto è molto formale, ma "data is" scorre meglio.

Versione Corretta (Consigliata) Ecco una versione che mantiene il tuo significato originale ma corregge gli errori e migliora la scorrevolezza:

"In recent years, cycling has become an increasingly popular and sustainable means of transport. However, cyclists often struggle to find accurate and updated information about bike paths. Existing data is usually scattered, incomplete, or not maintained, making it hard to evaluate the quality, safety, and usability of available routes. This lack of reliable information affects both occasional and regular cyclists, who may encounter poorly maintained paths, obstacles, or unsuitable routes without notice."

What we actually wrote: In recent years, cycling has become an increasingly popular and sustainable means of transportation. However, cyclists often face difficulties in finding accurate and up-to-date information about BPs. Ex- isting data are usually scattered, incomplete, or not maintained, making it hard to evaluate the quality, safety, and usability of available routes. This lack of reliable information affects both occasional and regular cyclists, who may encounter poorly maintained paths, obstacles, or unsuitable routes without prior notice.

Copyright © 2025, Federico Costa e Emanuele De Simone – All rights reserved